

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНТЕЛЕКТУАЛЬНИХ ТЕХНОЛОГІЙ**

**ПОЯСНЮВАЛЬНА ЗАПИСКА
ДО КУРСОВОЇ РОБОТИ З ПРОЕКТУВАННЯ АЛГОРИТМІВ
ТА ПРОГРАМУВАННЯ**

на тему:

«Графічна інтерпретація методу Монте-Карло для оцінки значення числа π »

Виконала студентка 2 курсу

групи КН-21

Катасонова Вероніка Юріївна

Засвідчую, що в курсовій роботі немає

запозичень з праць інших авторів

без відповідних посилань

Керівник курсової роботи

доцент Чолишкіна Ольга Геннадіївна

Оцінка за курсову роботу: _____

Члени комісії: _____

Київ - 2026

Київський національний університет імені Тараса Шевченка
Факультет інформаційних технологій
Кафедра інтелектуальних технологій

ЗАВДАННЯ

на курсову роботу з проектування алгоритмів та програмування

студентці **Катасоновій Вероніці Юріївні**

1. Тема роботи: “ Графічна інтерпретація методу Монте-Карло для оцінки значення числа π ”
2. Термін здачі закінченого проекту: 18 травня 2026 р.
3. Вихідні дані до проекту: немає
4. Зміст роботи:
 - Розділ 1. Аналіз методу Монте-Карло для оцінки значення числа π та формування вимог до програмного застосунку
 - Розділ 2. Проектування архітектури та графічного інтерфейсу застосунку для візуалізації методу Монте-Карло
 - Розділ 3. Реалізація програмного застосунку та дослідження точності оцінки значення числа π методом Монте-Карло
5. Перелік презентаційного матеріалу:
 1. Мета та завдання курсової роботи (1-2 слайди)
 2. Теоретичні основи методу Монте-Карло (1-2 слайди)
 3. Математична модель оцінки числа π (1 слайд)
 4. Архітектура програмного застосунку (2-3 слайди)
 5. Графічна візуалізація роботи алгоритму (2 слайди)
 6. Аналіз точності оцінки числа π залежно від кількості точок (2 слайди)
 7. Висновки (1 слайд)
6. Дата видачі завдання 23 лютого 2026 р.

Графік виконання курсової роботи

№	Назва етапу	Терміни	Примітки / відмітка про виконання
1	Вибір теми та керівника курсової роботи	23 лютого-1 березня	Заява на виконання курсової роботи, що підписана студентом керівником
2	Обговорення з керівником постановки завдання та змісту пояснювальної записки до курсової роботи	2 березня-8 березня	Заповнений бланк завдання на курсову роботу, що підписаний студентом та керівником роботи
3	Аналіз постановки задачі, формалізація задачі, вибір методів та засобів реалізації поставленої задачі, аналіз літературних джерел	9 березня-15 березня	Сформований матеріал до розділу 1 пояснювальної записки курсової роботи, оформлення списку джерел
4	<i>Перше узгодження з керівником</i>	16 березня-22 березня	
5	Розробка алгоритму, вибір структур даних, проектування програмного інтерфейсу з користувачем.	23 березня-29 березня	Сформований матеріал до розділу 2 пояснювальної
6	<i>Друге узгодження з керівником</i>	30 березня-5 квітня	
7	Розробка та тестування програмного продукту.	6 квітня-12 квітня	Готовий програмний продукт
8	<i>Демонстрація базового варіанту програмного продукту. Третє узгодження з керівником.</i>	13 квітня-19 квітня	
9	Доопрацювання програмного продукту, всебічне заключне тестування, розробка тестових прикладів та керівництва користувача.	20 квітня-26 квітня	Сформований матеріал до розділу 3 пояснювальної записки, підготовлений демонстраційний приклад роботи програми
10	Остаточне оформлення пояснювальної записки	27 квітня-3 травня	Готова пояснювальна записка та програмний застосунок
11	Подання роботи керівнику, перевірка роботи на плагіат, підготовка презентаційного матеріалу	4 травня-10 травня	Готова пояснювальна записка та програмний застосунок, результати перевірки на плагіат
12	Подання роботи до захисту	11 травня-17 травня	Готова пояснювальна записка, програмний застосунок, презентація для захисту курсової роботи, демонстраційний приклад роботи програми

Керівник роботи

Ольга ЧОЛИШКІНА

Завдання прийняла до виконання

Вероніка КАТАСОНОВА

ЗМІСТ

ВСТУП.....	4
1. АНАЛІЗ МЕТОДУ МОНТЕ-КАРЛО ДЛЯ ОЦІНКИ ЗНАЧЕННЯ ЧИСЛА π ТА ФОРМУВАННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАСТОСУНКУ.....	6
1.1 Загальна характеристика методу Монте-Карло.....	6
1.2 Використання методу Монте-Карло для оцінки числа π	7
1.3 Узагальнений алгоритм оцінки числа π методом Монте-Карло.....	9
1.4 Аналіз переваг та недоліків методу Монте-Карло.....	10
1.5 Формування вимог до програмного застосунку.....	11
1.6 Висновки до розділу 1.....	13
2. ПРОЕКТУВАННЯ АРХІТЕКТУРИ ТА ГРАФІЧНОГО ІНТЕРФЕЙСУ ЗАСТОСУНКУ ДЛЯ ВІЗУАЛІЗАЦІЇ МЕТОДУ МОНТЕ-КАРЛО.....	14
2.1 Функціональне проектування та узагальнений алгоритм роботи застосунку.....	14
2.2 Проектування графічного інтерфейсу користувача.....	21
2.3 Проектування структур даних, класів та алгоритму моделювання.....	24
2.4 Висновки до розділу 2.....	28
3. РЕАЛІЗАЦІЯ ПРОГРАМНОГО ЗАСТОСУНКУ ТА ДОСЛІДЖЕННЯ ТОЧНОСТІ ОЦІНКИ ЗНАЧЕННЯ ЧИСЛА π МЕТОДОМ МОНТЕ-КАРЛО.....	30
3.1 Обґрунтування вибору засобів розробки.....	30
3.2 Розробка структури програмного застосунку.....	30
3.3 Реалізація алгоритму та програмного застосунку.....	32
3.4 Тестування програмного застосунку.....	34
3.5 Експериментальні дослідження точності методу.....	40
3.6 Висновки до розділу 3.....	43
ВИСНОВКИ.....	44
ДЖЕРЕЛА ПОСИЛАННЯ.....	46
ДОДАТКИ.....	47

ВСТУП

У сучасних наукових та інженерних дослідженнях важливу роль відіграють методи чисельного моделювання, які дозволяють розв'язувати задачі, що складно або неможливо вирішити аналітичними методами. Одним із таких підходів є метод Монте-Карло, який є статистичним методом моделювання та використовується для обчислення очікуваних значень функцій, що не можуть бути інтегровані аналітично через відсутність замкнутої форми. Даний метод широко застосовується у різних галузях, зокрема у фізиці, математиці, інженерії, а також у фінансовому моделюванні [1].

Однією з класичних задач, що демонструє принцип роботи методу Монте-Карло, є оцінювання значення числа π . Генеруючи випадкові точки у межах одиничного квадрата та аналізуючи їх розташування відносно вписаного кола, можна отримати наближене значення числа π . Такий підхід дозволяє продемонструвати статистичну природу методу та залежність точності результату від кількості проведених експериментів.

Актуальність теми курсової роботи полягає у дослідженні можливостей використання методу Монте-Карло для оцінювання значення числа π , а також у створенні програмного застосунку, який дозволяє графічно візуалізувати процес випадкового розподілу точок та спостерігати вплив їх кількості на точність отриманого результату.

Метою курсової роботи є дослідження точності оцінки значення числа π методом Монте-Карло в залежності від кількості згенерованих точок, а також розробка програмного застосунку для графічної візуалізації процесу моделювання.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- проаналізувати теоретичні основи методу Монте-Карло;
- дослідити математичну модель оцінки числа π ;
- спроектувати програмний застосунок для реалізації алгоритму;
- реалізувати програму з використанням мови програмування Java та бібліотеки JavaFX;

- виконати тестування програми та проаналізувати точність отриманих результатів залежно від кількості згенерованих точок.

Об'єктом дослідження є методи статистичного моделювання.

Предметом дослідження є застосування методу Монте-Карло для оцінювання значення числа π .

Пояснювальна записка складається зі вступу, трьох розділів, висновків, переліку джерел посилання та додатків. У першому розділі розглянуто теоретичні основи методу Монте-Карло, математичну модель оцінювання числа π та сформульовано вимоги до програмного застосунку. У другому розділі виконано проектування функціональної структури, графічного інтерфейсу, класів і алгоритму моделювання. У третьому розділі описано реалізацію програмного застосунку, проведено його тестування та експериментальне дослідження точності методу.

Для реалізації програмного застосунку використано мову програмування Java, бібліотеку JavaFX для створення графічного інтерфейсу та візуалізації процесу моделювання, систему Maven для організації проекту, а також середовище розробки IntelliJ IDEA.

Практичне значення одержаних результатів полягає у створенні працездатного програмного застосунку, який дозволяє наочно демонструвати роботу методу Монте-Карло для оцінювання числа π , виконувати покрокову та автоматичну візуалізацію моделювання, а також досліджувати залежність похибки обчислення від кількості згенерованих точок.

1 АНАЛІЗ МЕТОДУ МОНТЕ-КАРЛО ДЛЯ ОЦІНКИ ЗНАЧЕННЯ ЧИСЛА π ТА ФОРМУВАННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАСТОСУНКУ

1.1 Загальна характеристика методу Монте-Карло

Метод Монте-Карло - це чисельний метод розв'язання різних математичних задач, в яких параметри моделюються у формі випадкових величин. Даний метод оснований на комп'ютерній імітації розподілів цих величин та подальшому аналізі отриманих результатів. Він являє собою імітаційний метод, який історично отримав свою назву від міста Монте-Карло, відомого своїми гральними будинками і казино [2].

Метод Монте-Карло тісно пов'язаний із завданнями теорії ймовірностей, математичної статистики й обчислювальної математики. У зв'язку із завданням моделювання випадкових величин (особливо рівномірно розподілених) важливу роль відіграють також методи генерації випадкових чисел.

Даний метод призначений для розв'язання широкого кола задач, особливо тих, що допускають імовірнісний опис процесів. Оскільки метод Монте-Карло потребує проведення великої кількості випробувань, його часто називають методом статистичних випробувань [3].

Основою методу Монте-Карло є використання генератора випадкових чисел, який формує послідовність випадкових величин. Вони зазвичай є незалежними та однаково розподіленими відповідно до заданого закону розподілу.

У більшості задач використовуються псевдовипадкові числа, що генеруються спеціальними алгоритмами та мають статистичні властивості, близькі до істинно випадкових значень. Це дозволяє застосовувати метод Монте-Карло для моделювання різних випадкових процесів [4].

1.1.1 Класифікація методів Монте-Карло

Методи Монте-Карло класифікуються залежно від способу організації обчислювального процесу та характеру використання випадкових величин.

Найпростішим є метод прямого статистичного моделювання, який базується на генерації незалежних випадкових величин із заданим розподілом. У цьому підході результат оцінюється шляхом багаторазового проведення незалежних випробувань і статистичного узагальнення отриманих значень. Саме цей метод використовується для оцінювання значення числа π , оскільки дозволяє безпосередньо пов'язати геометричну ймовірність потрапляння точки в область із шуканою величиною.

Більш складним підходом є методи на основі марківських ланцюгів (МСМС), у яких випадкові величини вже не є незалежними, а формують послідовність залежних станів. Такі методи застосовуються для моделювання складних розподілів, де пряма генерація випадкових величин є утрудненою або неможливою.

Окрему групу становлять методи зменшення дисперсії, які спрямовані на підвищення точності результатів без значного збільшення кількості випробувань. До них належать, зокрема, метод важливого вибору, стратифікована вибірка та інші підходи, що дозволяють ефективніше використовувати згенеровані випадкові дані.

Таким чином, вибір конкретного підходу методу Монте-Карло визначається складністю задачі, вимогами до точності результатів та доступними обчислювальними ресурсами [4].

1.2 Використання методу Монте-Карло для оцінки числа π

У XVIII столітті Жорж Луї Леклерк де Бюффон сформулював задачу про знаходження ймовірності того, що голка, яку кидають на лист розграфленого паперу, перетинає одну з ліній. Виявилося, що ця ймовірність пов'язана з числом π , і це зробило можливим пошук цього числа ймовірнісними методами, а саме, методом Монте-Карло [5].

Зараз оцінка числа π методом Монте-Карло здійснюється за допомогою генерації випадкових точок у межах квадрата та визначення кількості точок, що потрапляють у вписане в нього коло. Ймовірність потрапляння точки в коло пропорційна відношенню площі кола до площі квадрата.

Для визначення належності точки до кола використовується рівняння кола. У загальному випадку, якщо центр кола розташований у початку координат, умова має вигляд:

$$x^2 + y^2 \leq r^2.$$

Однак якщо використовується одиничний квадрат $[0;1] \times [0;1]$, у який вписане коло з центром у точці $(0,5; 0,5)$ та радіусом $0,5$, для перевірки належності точки до кола використовується умова:

$$(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25.$$

Якщо ця умова виконується, то точка вважається такою, що лежить усередині кола.

Наближене значення числа π визначається за формулою:

$$\pi \approx 4 \cdot \left(\frac{k}{N}\right),$$

де k - кількість точок у колі, N - загальна кількість точок [6].

Оскільки площа круга дорівнює πr^2 , а площа квадрата $= (2r)^2 = 4r^2$, то їх відношення дорівнює $\pi/4$. Саме тому частка точок, що потрапили в коло, приблизно дорівнює $\pi/4$. Помноживши це значення на 4, отримуємо оцінку числа π .

Чим більша кількість згенерованих точок, тим точнішою є отримана оцінка значення числа π , що пояснюється статистичною природою методу Монте-Карло. Даний підхід є геометричною інтерпретацією даного методу.

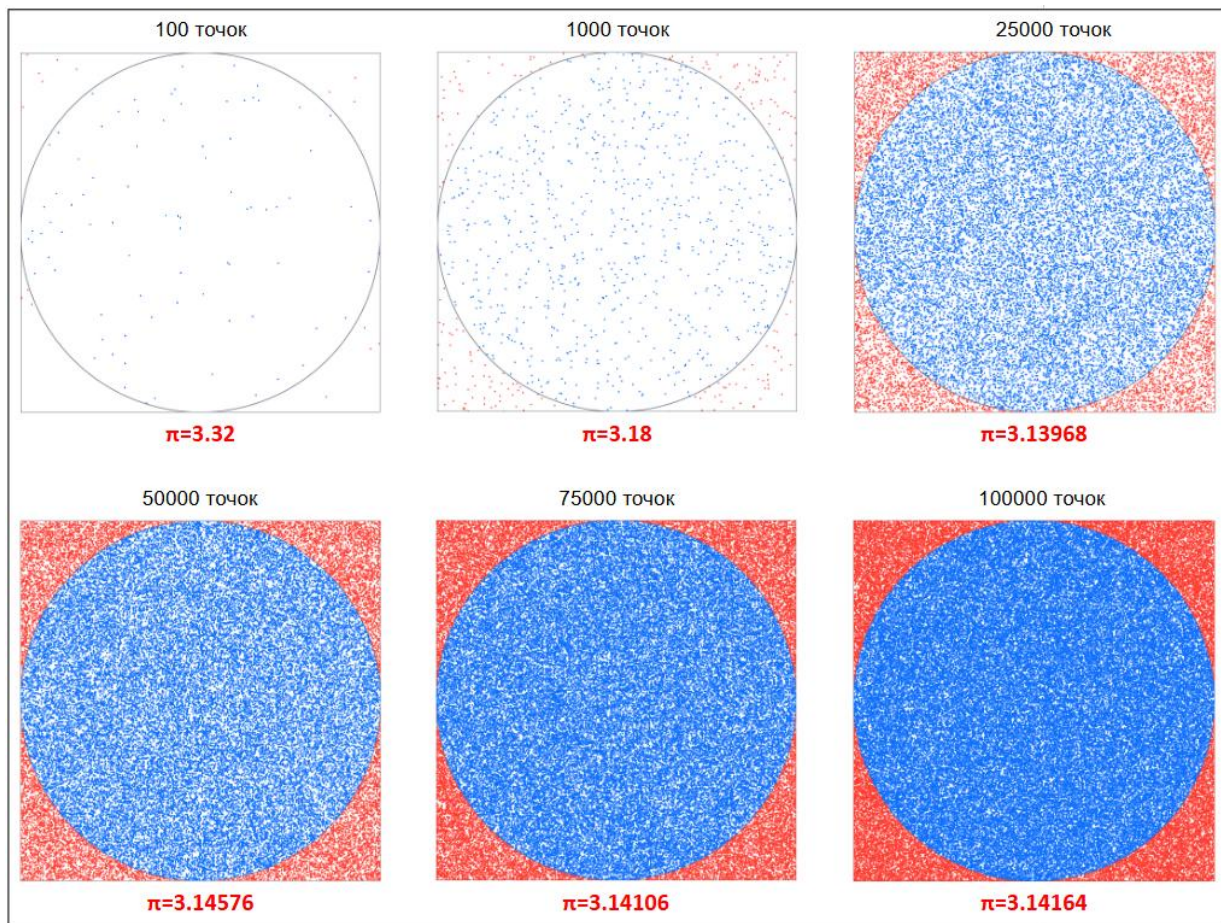


Рисунок 1.1 - Залежність точності методу Монте-Карло від кількості згенерованих точок

На рисунку 1.1 показано ансамбль скріншотів авторської програми, що відображає залежність точності оцінки значення числа π від кількості згенерованих випадкових точок. Зі збільшенням кількості точок спостерігається зменшення похибки та наближення обчисленого значення до точного значення числа π . Це підтверджує статистичну природу методу Монте-Карло та залежність точності результату від обсягу вибірки.

1.3 Узагальнений алгоритм оцінки числа π методом Монте-Карло

Метод Монте-Карло не має єдиного алгоритму - він є підходом, який можна реалізувати різними способами, залежно від задачі. Алгоритм оцінки числа π методом Монте-Карло можна представити у вигляді кроків:

- задати кількість випадкових точок N ;

- ініціалізувати лічильник точок, що потрапили в коло ($k = 0$);
- для кожної точки згенерувати випадкові координати (x, y) у межах квадрата;
- перевірити умову належності точки до кола: $(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25$;
- якщо умова виконується, збільшити лічильник k на 1;
- повторити кроки 3-5 для всіх N точок;
- обчислити значення числа π за формулою:

$$\pi \approx 4 * \frac{k}{N}.$$

Отже, алгоритм дозволяє отримати наближене значення числа π , точність якого залежить від кількості згенерованих точок.

1.4 Аналіз переваг та недоліків методу Монте-Карло

Метод Монте-Карло має досить багато переваг, що є причиною його широкого використання в різних галузях. Він дозволяє враховувати невизначеність вхідних параметрів за рахунок використання випадкових величин, що робить моделювання досліджуваних процесів реалістичнішим.

Важливою перевагою є також можливість кількісної оцінки ризику та аналізу різних варіантів розвитку подій. Метод дозволяє досліджувати вплив змін вхідних параметрів на кінцевий результат, що робить його ефективним інструментом для аналізу чутливості [7].

Ще однією перевагою методу є універсальність, бо він може застосовуватись до широкого класу задач, у тому числі для обчислення складних функцій та інтегралів, для яких відсутній аналітичний розв'язок.

Метод Монте-Карло також дозволяє отримувати досить точні результати, але за умови використання великої кількості випробувань. Чим більшою є кількість згенерованих точок, тим вищою буде точність оцінки [8].

Але метод має і певні недоліки. Одним із основних є залежність результатів від припущень щодо розподілів випадкових величин. Неправильний вибір цих припущень може призвести до неточних результатів.

Також метод потребує багато обчислювальних ресурсів, оскільки для отримання високої точності результату необхідно виконувати велику кількість експериментів. Інтерпретація отриманих результатів потребує певних знань у галузі теорії ймовірностей і математичної статистики, що ускладнює його застосування [7].

Отже, метод Монте-Карло є ефективним і універсальним інструментом, проте його застосування потребує обережності при виборі параметрів моделі та достатніх обчислювальних ресурсів.

1.5 Формування вимог до програмного застосунку

Для реалізації методу Монте-Карло для оцінювання значення числа π необхідно сформулювати вимоги до програмного застосунку. Програма повинна забезпечувати зручний інтерфейс користувача, візуалізацію процесу моделювання та коректну реалізацію алгоритму.

Основні функціональні вимоги:

- можливість введення кількості випадкових точок, що використовуються для оцінювання значення числа π у межах від 100 до 100000;
- можливість вибору режиму візуалізації процесу моделювання (анімований або автоматичний);
- можливість встановлення швидкості анімованої візуалізації;
- можливість покрокового виконання алгоритму з відображенням результату кожної ітерації;
- генерація випадкових координат точок у межах одиничного квадрата;
- перевірка належності точок до кола за умовою $(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25$;
- накопичення та оновлення статистичних даних у процесі моделювання;
- обчислення наближеного значення числа π ;

- обчислення статистичних показників, в тому числі і похибки обчислення;
- графічна візуалізація процесу моделювання, включаючи відображення геометричної області, згенерованих точок та результатів їх класифікації;
- відображення результатів обчислення користувачу;
- можливість керування виконанням алгоритму: запуск моделювання, призупинення та відновлення анімованої візуалізації, покрокове виконання однієї ітерації після призупинення, скидання результатів;
- можливість збереження результатів моделювання для подальшого аналізу;
- відображення графіка залежності похибки від кількості точок.

Також програмний застосунок повинен забезпечувати коректність обчислень та стабільність роботи при різній кількості точок.

До нефункціональних вимог належать:

- забезпечення зручного графічного інтерфейсу користувача для введення параметрів моделювання та керування виконанням алгоритму;
- забезпечення наочного відображення процесу генерації та класифікації випадкових точок;
- підтримка покрокового спостереження за виконанням алгоритму в анімованому режимі;
- забезпечення коректної роботи застосунку при кількості точок у межах від 100 до 100000;
- забезпечення достатньої швидкості виконання обчислень в автоматичному режимі моделювання.

Сформульовані вимоги визначають функціональні можливості програмного застосунку та забезпечують узгодженість між поставленою задачею, алгоритмом її розв'язання і програмною реалізацією.

1.6 Висновки до розділу 1

У даному розділі було розглянуто теоретичні основи методу Монте-Карло, що є поширеним методом статистичного моделювання. Визначено основні принципи його роботи, що базуються на використанні випадкових величин та повторюваних експериментів.

Проаналізовано застосування методу для оцінювання значення числа π , що базується на геометричній інтерпретації та використанні випадкових точок у межах квадрата. Розглянуто узагальнений алгоритм реалізації методу, який дозволяє отримати наближене значення числа π .

Також було досліджено основні переваги та недоліки методу Монте-Карло, що допомагає оцінити ефективність його використання та обмеження; було сформульовано вимоги до програмного застосунку, який буде реалізовано у наступних розділах курсової роботи.

Отримані результати теоретичного аналізу є основою для подальшого проектування та реалізації програмного застосунку для графічної інтерпретації методу Монте-Карло.

2 ПРОЕКТУВАННЯ АРХІТЕКТУРИ ТА ГРАФІЧНОГО ІНТЕРФЕЙСУ ЗАСТОСУНКУ ДЛЯ ВІЗУАЛІЗАЦІЇ МЕТОДУ МОНТЕ-КАРЛО

2.1 Функціональне проектування та узагальнений алгоритм роботи застосунку

На основі сформульованих у розділі 1 функціональних та нефункціональних вимог виконується функціональне проектування програмного застосунку. Метою даного етапу є визначення основних функцій системи, їх структуру та встановлення взаємозв'язків між ними.

Програмний застосунок призначений для графічної інтерпретації методу Монте-Карло для оцінювання значення числа π . Його робота основана на генерації випадкових точок у межах заданої області, перевірці їх належності до кола та обчисленні наближеного значення числа π на основі статистичних даних.

Відповідно до поставлених вимог визначено перелік основних функцій програмного застосунку, які забезпечують виконання моделювання, обробку результатів та їх візуалізацію:

- Ф1 Ініціалізація параметрів моделювання
 - Ф1.1 Введення кількості точок. Програма забезпечує введення користувачем кількості випадкових точок N , що визначає обсяг статистичного експерименту; N має бути додатним цілим числом у межах від 100 до 100000, у разі некоректного введення виводиться повідомлення про помилку.
 - Ф1.2 Встановлення швидкості анімованої візуалізації. Програма забезпечує встановлення швидкості відображення процесу моделювання в анімованому режимі в межах допустимого діапазону значень.
- Ф2 Генерація та обробка випадкових точок
 - Ф2.1 Генерація випадкових координат точки. Програма генерує координати точки (x, y) у межах одиничного квадрата, де $x, y \in [0;1]$.

- Ф2.2 Перевірка належності точки до кола. Програма визначає, чи належить точка колу, вписаному в одиничний квадрат. Центр кола розташований у точці (0,5; 0,5), радіус дорівнює 0,5, тому перевірка виконується за умовою:

$$(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25.$$
- Ф2.3 Оновлення статистичних даних. Програма накопичує статистичні дані, зокрема загальну кількість точок та кількість точок, що потрапили до кола.
- Ф3 Візуалізація процесу моделювання
 - Ф3.1 Відображення геометричної області. Програма відображає одиничний квадрат та вписане в нього коло для наочного представлення області моделювання.
 - Ф3.2 Відображення згенерованих точок. Програма відображає кожну згенеровану точку на координатній площині.
 - Ф3.3 Відображення результату класифікації точки. Програма відображає точки різними кольорами залежно від їх належності до кола.
- Ф4 Обчислення та відображення результатів
 - Ф4.1 Обчислення значення числа π . Програма обчислює наближене значення числа π за формулою $\pi \approx 4 * \frac{k}{N}$, де k - кількість точок у колі, N - загальна кількість точок.
 - Ф4.2 Відображення результатів обчислення. Програма відображає значення π та статистичні дані користувачу у текстовому вигляді.
- Ф5 Керування виконанням алгоритму
 - Ф5.1 Запуск моделювання. Програма забезпечує запуск процесу моделювання на основі заданих параметрів. Після запуску в автоматичному режимі моделювання виконується до завершення без можливості паузи або покрокового керування. В анімованому режимі після запуску користувачу доступне керування процесом візуалізації.

- Ф5.2 Вибір режиму візуалізації. Програма забезпечує вибір режиму відображення: автоматичного (миттєва генерація точок) або анімованого (поступове відображення точок).
- Ф5.3 Керування анімованою візуалізацією. Програма забезпечує керування виконанням алгоритму в анімованому режимі. Під час запуску анімованого режиму користувач може призупинити моделювання, після призупинення - продовжити виконання або виконувати алгоритм покроково.
- Ф5.4 Покрокове виконання алгоритму. Програма забезпечує виконання однієї ітерації моделювання після призупинення анімованої візуалізації.
- Ф5.5 Скидання результатів. Програма забезпечує очищення графічної області та обнулення статистичних даних. Скидання доступне після призупинення анімованого моделювання або після завершення моделювання.
- Ф6 Збір статистичних даних та аналіз результатів
 - Ф6.1 Збереження результатів моделювання. Програма забезпечує збереження результатів (кількість точок, значення π , статистичні дані) для подальшого аналізу.
 - Ф6.2 Обчислення статистичних показників. Програма обчислює похибку $\Delta = |\pi_{\text{обчислене}} - \pi_{\text{точне}}|$, де $\pi_{\text{точне}} \approx 3.1415926535$.
 - Ф6.3 Відображення графіка залежності похибки від кількості точок. Програма забезпечує можливість порівняння результатів декількох запусків моделювання, а саме залежність похибки обчислення числа π від кількості встановлених точок. Результати порівняння представлені у вигляді графіка залежності.

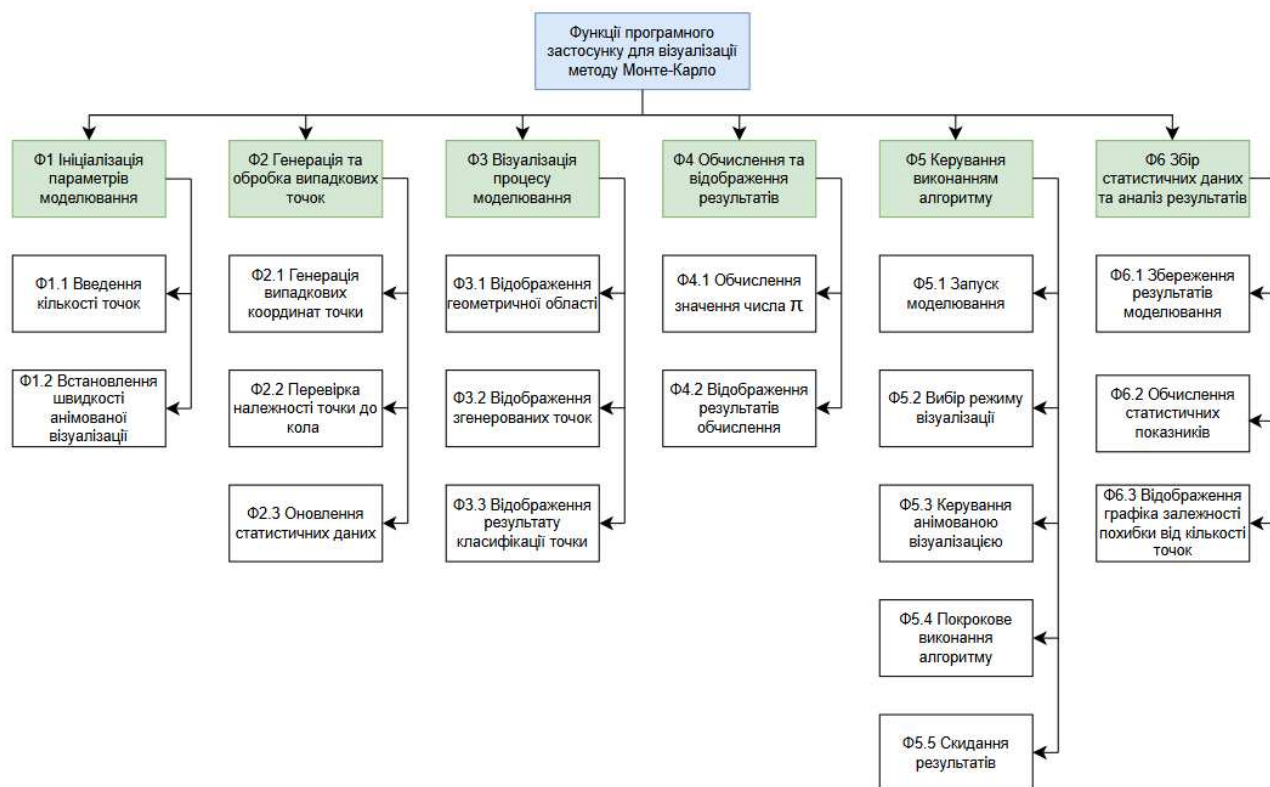


Рисунок 2.1 - Функції програмного застосунку для візуалізації методу Монте-Карло у вигляді дерева функцій

На рисунку 2.1 зображено функціональну структуру програмного застосунку у вигляді дерева функцій. У межах цієї структури визначено основні функції системи та їх підфункції.

Виділені функції охоплюють усі ключові аспекти роботи застосунку, зокрема ініціалізацію параметрів моделювання, генерацію та обробку випадкових точок, візуалізацію процесу, обчислення результатів, керування виконанням алгоритму, а також збір і аналіз статистичних даних.

Представлення функцій у вигляді дерева дозволяє систематизувати функціональні можливості застосунку та визначити їх взаємозв'язок і рівень деталізації.

На основі побудованого дерева функцій здійснюється подальша формалізація взаємодії користувача із системою у вигляді UML-діаграми варіантів використання.

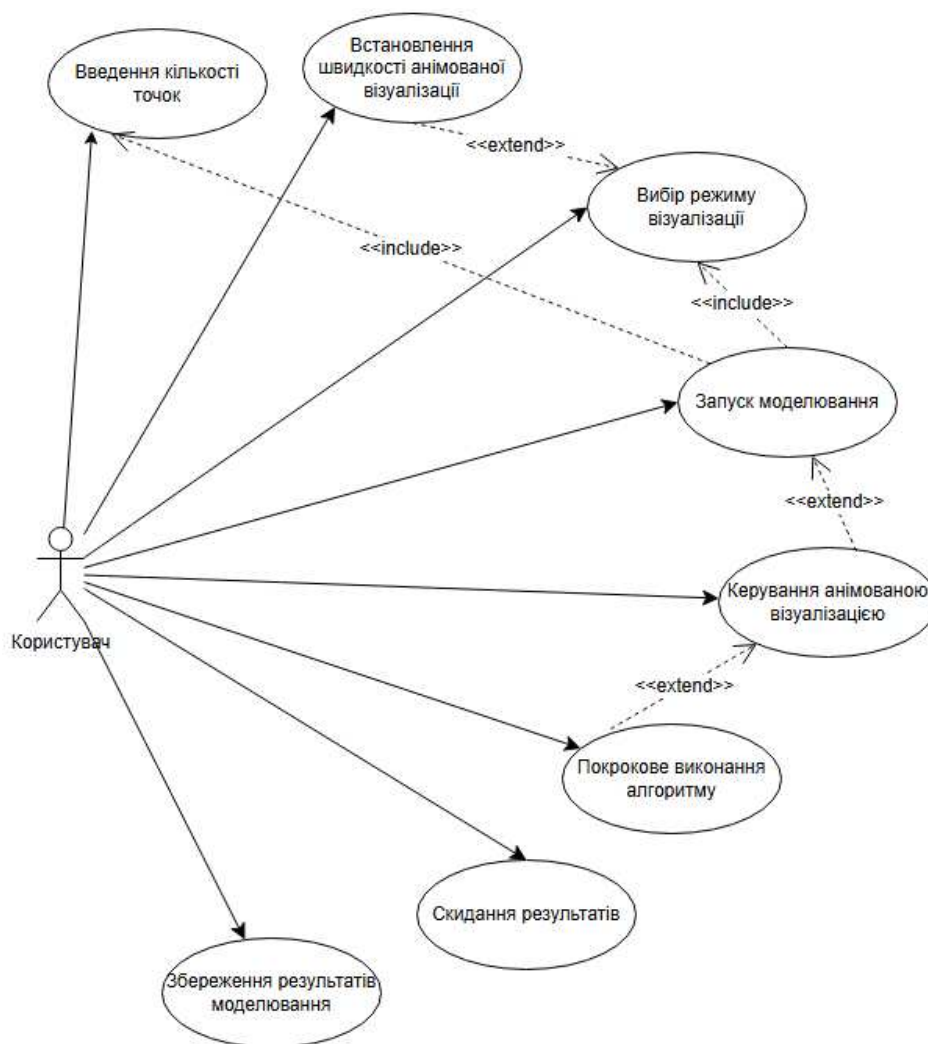


Рисунок 2.2 - Функціональний аналіз програмного застосунку для візуалізації методу Монте-Карло у вигляді діаграми варіантів використання

На рисунку 2.2 наведено функціональний аналіз програмного застосунку у вигляді діаграми варіантів використання. Основним актором системи є користувач, який взаємодіє із застосунком з метою задання параметрів моделювання та керування процесом його виконання.

Користувач має можливість вводити кількість випадкових точок, встановлювати швидкість анімованої візуалізації, обирати режим відображення процесу моделювання, запускати моделювання, керувати виконанням анімації, виконувати покрокове виконання алгоритму, а також виконувати збереження та скидання результатів.

Між варіантами використання встановлено зв'язки типу <<include>> та <<extend>>. Зв'язки <<include>> використовуються для відображення

обов'язкових дій, що входять до складу сценарію запуску моделювання. Зокрема, варіант використання «Запуск моделювання» включає «Введення кількості точок» та «Вибір режиму візуалізації», оскільки ці дії є необхідними для коректного виконання процесу.

Зв'язки <<extend>> відображають додаткову функціональність, що виконується за певних умов. Варіант використання «Керування анімованою візуалізацією» розширює «Запуск моделювання» і використовується під час виконання моделювання в анімованому режимі. Так само, «Встановлення швидкості анімованої візуалізації» розширює «Вибір режиму візуалізації» і застосовується лише для анімованого режиму. Варіант використання «Покрокове виконання алгоритму» розширює «Керування анімованою візуалізацією», оскільки доступний після призупинення анімованої візуалізації та забезпечує виконання однієї наступної ітерації алгоритму.

Окремо виділено варіанти використання, що пов'язані з роботою з результатами: «Збереження результатів моделювання» забезпечує збереження отриманих даних для подальшого аналізу, а «Скидання результатів» дозволяє повернути застосунок до початкового стану для проведення нового експерименту.

Після побудови діаграми варіантів використання необхідно розглянути узагальнений алгоритм роботи застосунку, який відображає послідовність дій користувача та реакцій системи на них.

На початку роботи користувач вводить кількість випадкових точок, обирає режим візуалізації та, за необхідності, встановлює швидкість анімації. Після цього запускається процес моделювання, під час якого відображається геометрична область та здійснюється генерація випадкових точок.

Для кожної точки визначається її належність до кола та оновлюються статистичні дані. В анімованому режимі точки відображаються поступово з можливістю паузи та покрокового виконання, тоді як в автоматичному режимі обробка виконується без покрокового відображення.

Після завершення генерації точок обчислюється наближене значення числа π , визначаються статистичні показники та відображаються результати. Користувач

може зберегти результати для оновлення графіка залежності похибки від кількості точок або скинути їх для проведення нового експерименту.

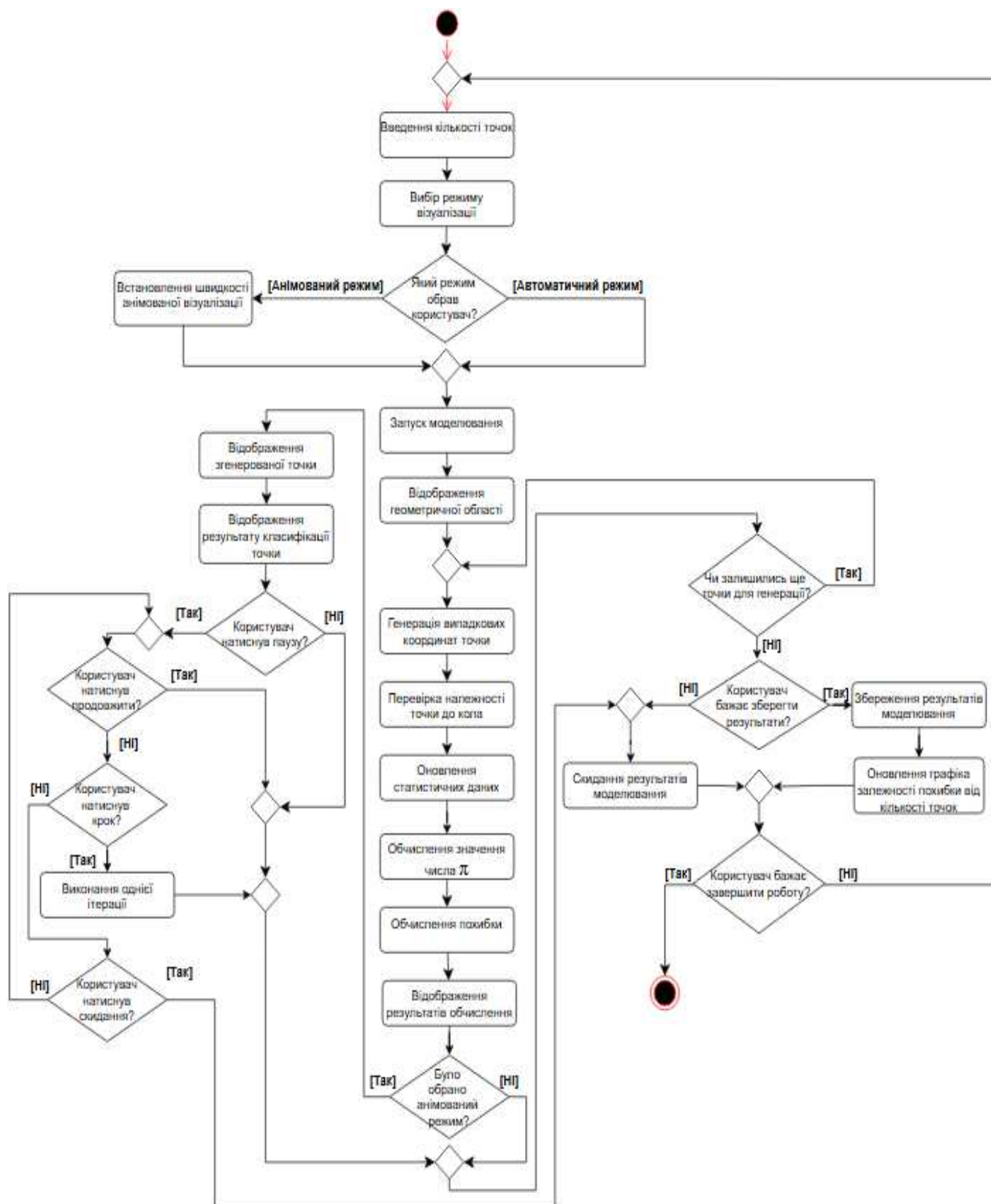


Рисунок 2.3 - Узагальнений алгоритм роботи програмного застосунку для візуалізації методу Монте-Карло у вигляді UML-діаграми діяльності

На рисунку 2.3 наведено узагальнений алгоритм роботи програмного застосунку у вигляді UML-діаграми діяльності. Діаграма відображає послідовність дій користувача та реакцій системи під час виконання моделювання, включаючи введення параметрів, вибір режиму візуалізації, генерацію і обробку випадкових точок, а також обчислення та відображення результатів.

У діаграмі враховано розгалуження залежно від обраного режиму роботи (анімований або автоматичний) і в анімованому режимі передбачено можливість призупинення моделювання. Після призупинення користувач може продовжити автоматичне анімоване виконання, виконати одну наступну ітерацію за допомогою кнопки «Крок» або скинути результати моделювання. В автоматичному режимі моделювання виконується без паузи та покрокового керування. Після завершення моделювання передбачено виконання додаткових дій користувача, зокрема збереження або скидання результатів чи завершення роботи застосунку.

2.2 Проектування графічного інтерфейсу користувача

Проектування графічного інтерфейсу користувача необхідне для забезпечення зручної взаємодії користувача з програмним застосунком під час виконання моделювання методом Монте-Карло.

Інтерфейс розробляється з урахуванням визначених варіантів використання та функцій системи, що забезпечує узгодженість між можливостями застосунку та засобами їх реалізації. Особлива увага приділяється підтримці покрокової анімованої візуалізації, а також засобам керування алгоритму, тобто запуску, паузі та зміні швидкості анімації.

ПАНЕЛЬ КЕРУВАННЯ

ВІЗУАЛІЗАЦІЯ

РЕЗУЛЬТАТИ ОБЧИСЛЕННЯ

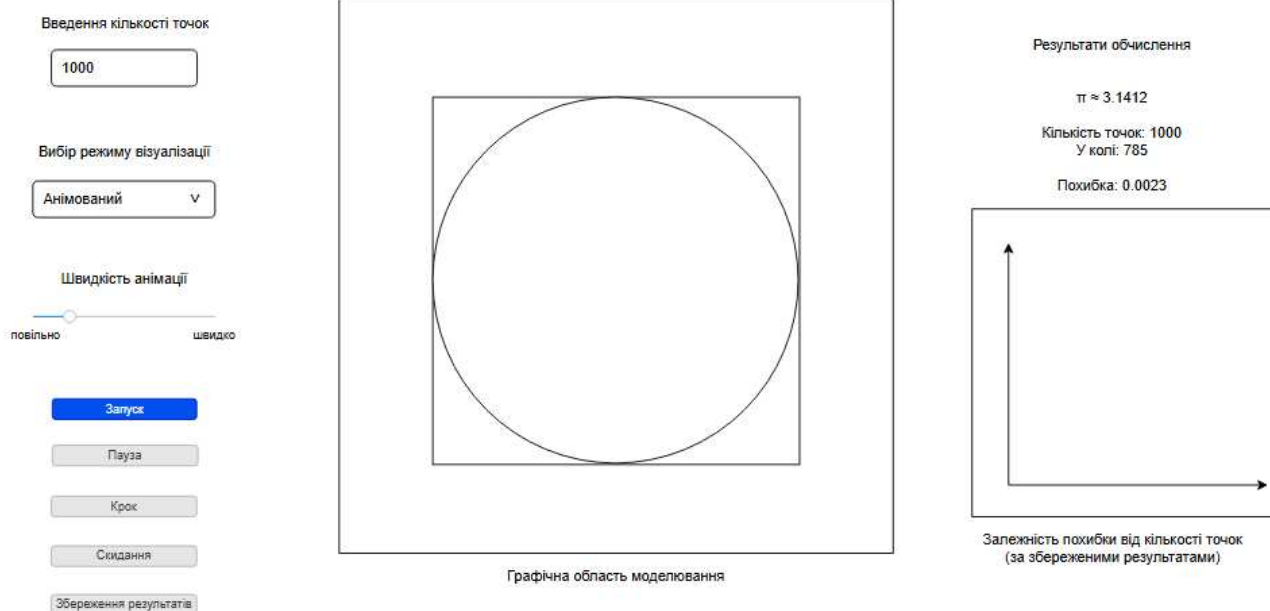


Рисунок 2.4 - Прототип головного вікна програмного застосунку

На рисунку 2.4 наведено графічний прототип головного вікна програмного застосунку для візуалізації методу Монте-Карло. Інтерфейс побудований відповідно до визначених варіантів використання та забезпечує зручну взаємодію користувача з системою під час виконання моделювання.

Головне вікно складається з трьох основних частин: панелі керування, графічної області моделювання та області відображення результатів обчислення. Панель керування містить елементи для введення кількості точок, вибору режиму візуалізації, встановлення швидкості анімованої візуалізації, а також кнопки керування виконанням алгоритму (запуск, пауза/продовження, крок, скидання, збереження результатів).

Стан кнопок змінюється залежно від етапу роботи застосунку. До запуску моделювання активною є лише кнопка «Запуск». В автоматичному режимі після запуску моделювання виконується до завершення, після чого доступними стають кнопки «Скидання» та «Збереження результатів». В анімованому режимі після запуску активною стає кнопка «Пауза». Після призупинення моделювання користувач може продовжити виконання, виконати один крок алгоритму або

скинути результати. Після завершення анімованого моделювання доступними залишаються кнопки «Скидання» та «Збереження результатів».

Графічна область моделювання призначена для відображення процесу генерації випадкових точок у межах одиничного квадрата та їх класифікації відносно кола. У цій області користувач може спостерігати покрокове виконання алгоритму в анімованому режимі або миттєве відображення результату в автоматичному режимі.

Область результатів обчислення містить інформацію про наближене значення числа π , загальну кількість згенерованих точок, кількість точок, що потрапили до кола, а також значення похибки обчислення. Також у області результатів передбачено відображення графічної залежності похибки від кількості точок, який оновлюється після кожного збереження результатів моделювання.

Сценарій взаємодії користувача з інтерфейсом складається з введення кількості точок, вибору режиму візуалізації та, за необхідності, встановлення швидкості анімації. Після цього користувач запускає моделювання, у процесі якого може призупиняти та відновлювати анімовану візуалізацію. Після завершення моделювання користувач отримує результати, які можуть бути збережені або скинуті для проведення нового експерименту. У разі збереження результатів, до графіка залежності додаватиметься нова точка для порівняльного аналізу похибки обчислення від кількості згенерованих точок.

Робота інтерфейсу безпосередньо пов'язана з функціями програмного застосунку, визначеними у підрозділі 2.1, зокрема: Ф1.1 - введення кількості точок, Ф1.2 - встановлення швидкості анімованої візуалізації, Ф3.1-Ф3.3 - візуалізація процесу моделювання, Ф4.1-Ф4.2 - обчислення та відображення результатів, Ф5.1-Ф5.5 - керування виконанням алгоритму, Ф6.1-Ф6.3 - збір статистичних даних та аналіз результатів.

2.3 Проектування структур даних, класів та алгоритму моделювання

2.3.1 Проектування структур даних

Для реалізації методу Монте-Карло у програмному застосунку використовуються прості структури даних, що забезпечують зберігання параметрів моделювання та результатів обчислення.

Основними даними є координати випадкових точок, загальна кількість точок, кількість точок, що потрапили до кола, а також обчислене значення числа π та похибка.

Координати точок подаються у вигляді пар чисел (x, y) , де $x, y \in [0;1]$. Для накопичення статистичних даних використовуються змінні-лічильники, які зберігають загальну кількість згенерованих точок та кількість точок, що належать до кола.

2.3.2 Проектування класів

У процесі проектування структур даних було визначено основні класи програмного застосунку, які відображають сутності предметної області та забезпечують реалізацію методу Монте-Карло.

- Клас `Point` представляє випадкову точку з координатами та містить метод для перевірки її належності до кола.
- Клас `SimulationParameters` призначений для зберігання параметрів моделювання, зокрема кількості точок, швидкості анімації та режиму візуалізації. Для обмеження можливих значень режиму використовується перелічуваний тип `VisualizationMode`.
- Клас `Statistics` відповідає за накопичення статистичних даних під час моделювання, а також обчислення наближеного значення числа π та похибки.

- Клас `IterationState` використовується для представлення поточного стану виконання алгоритму, що дозволяє реалізувати покрокову анімацію процесу моделювання.
- Клас `SimulationResult` представляє результат одного запуску моделювання та містить обчислені значення, зокрема оцінку числа π і похибку.
- Клас `ResultHistory` забезпечує збереження результатів кількох запусків моделювання для здійснення подальшого аналізу та побудови залежності похибки від кількості точок.
- Клас `MonteCarloSimulation` виконує основну логіку алгоритму та відповідає за генерацію випадкових точок, оновлення статистики та формування результатів. Даний клас реалізує інтерфейс `IMonteCarloAlgorithm`, що визначає спільний набір методів для запуску, покрокового виконання та завершення моделювання.

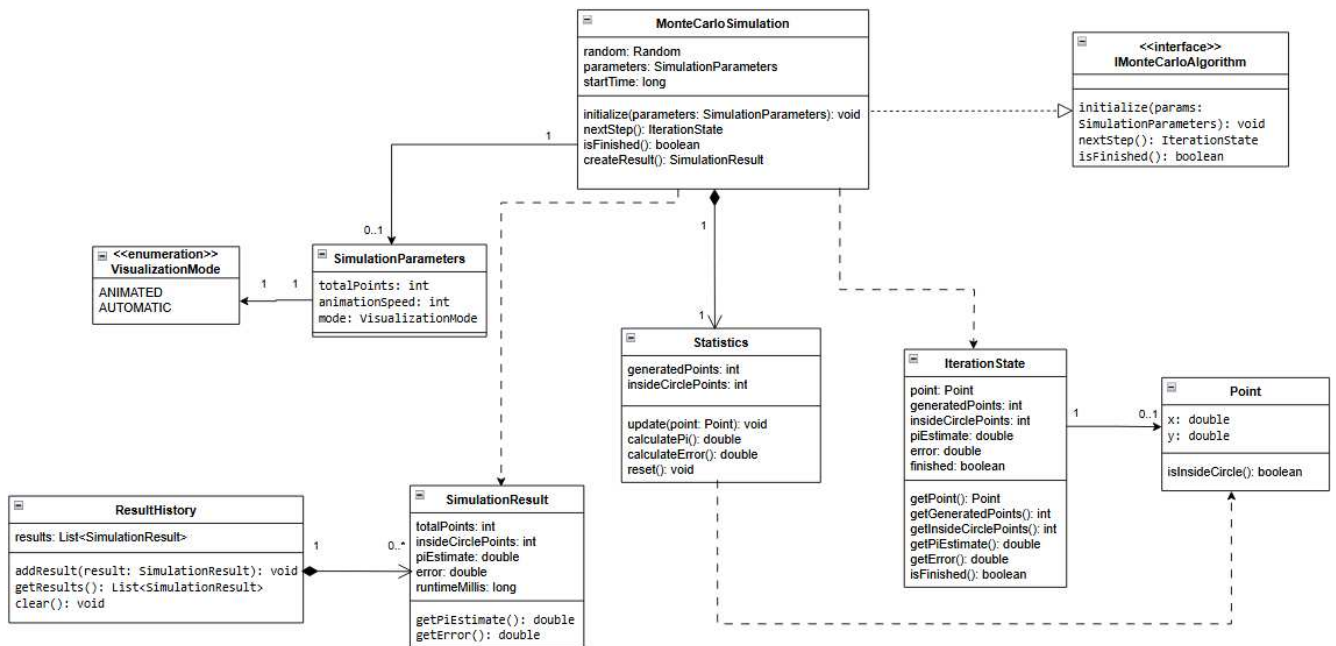


Рисунок 2.5 - UML-діаграма класів програмного застосунку

На рисунку 2.5 наведено UML-діаграму класів, що відображає структуру програмного застосунку та взаємозв'язки між основними компонентами. Дана структура класів забезпечує розділення відповідальностей, підтримує покрокове

виконання алгоритму та дозволяє розширювати функціональність програмного застосунку без зміни його основної логіки.

2.3.3 Проектування алгоритму моделювання

Алгоритм моделювання реалізується у вигляді методів класу `MonteCarloSimulation`, який реалізує інтерфейс `IMonteCarloAlgorithm`, та забезпечує генерацію випадкових точок, перевірку їх належності до кола і обчислення наближеного значення числа π .

На початковому етапі виконується ініціалізація параметрів моделювання за допомогою методу `initialize(parameters: SimulationParameters)`, у якому встановлюється кількість точок, режим візуалізації та інші параметри.

Основний процес моделювання реалізовано за допомогою методу `nextStep(): IterationState`, один виклик якого відповідає одній ітерації алгоритму. В анімованому режимі метод `nextStep()` викликається послідовно із затримкою між ітераціями, що забезпечує поступове відображення точок. В автоматичному режимі цей самий метод викликається у циклі без затримки до досягнення заданої кількості точок.

Під час одного виклику методу `nextStep(): IterationState` виконуються такі дії:

- перевірка ініціалізації параметрів моделювання;
- перевірка завершення моделювання;
- генерація випадкової точки `Point`;
- оновлення статистичних даних за допомогою методу `update(point: Point)` класу `Statistics`;
- перевірка належності точки до кола за допомогою методу `isInsideCircle()` за умови: $(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25$;
- формування об'єкта `IterationState`, що містить поточну точку, кількість згенерованих точок, кількість точок у колі, поточне значення π , похибку та ознаку завершення моделювання.

Для завершення моделювання після досягнення заданої кількості точок формується об'єкт `SimulationResult`, який містить загальну кількість точок, кількість точок у колі, обчислене значення числа π , похибку та час виконання моделювання.

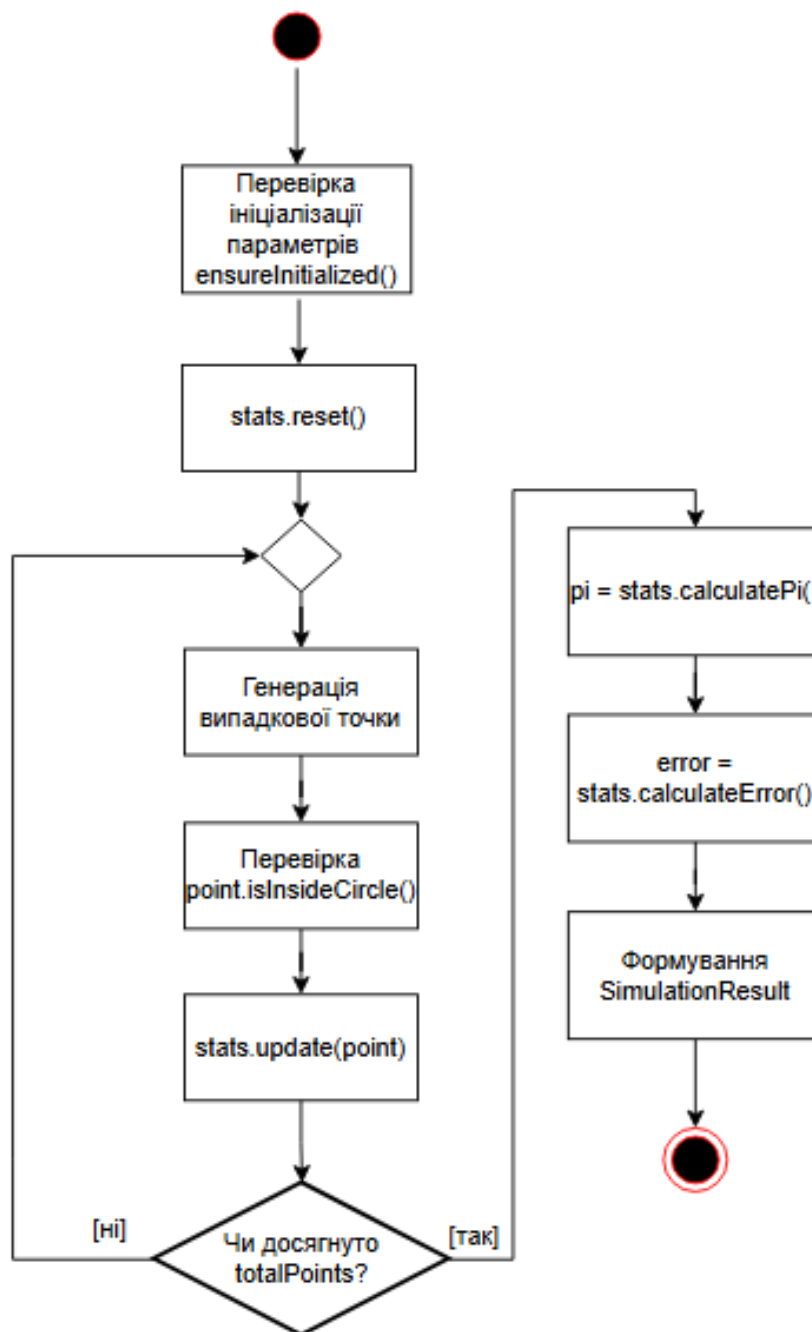


Рисунок 2.6 - Алгоритм моделювання у класі `MonteCarloSimulation` у вигляді блок-схеми

Блок-схема на рисунку 2.6 відображає загальну логіку виконання моделювання у класі `MonteCarloSimulation`. Перед початком виконання алгоритму

параметри моделювання мають бути ініціалізовані методом `initialize(parameters: SimulationParameters)`, після чого статистичні дані скидаються.

Далі виконується ітераційний процес генерації випадкових точок. Для кожної точки створюються координати в межах одиничного квадрата, виконується перевірка її належності до кола методом `isInsideCircle()`, після чого статистичні дані оновлюються методом `update(point: Point)`. Цикл повторюється доти, доки кількість згенерованих точок не досягне значення `totalPoints`.

Після завершення ітераційного процесу обчислюється наближене значення числа π , визначається похибка обчислення та формується об'єкт `SimulationResult`, який містить підсумкові результати моделювання.

Для проведення обчислювальних експериментів передбачено використання класу `ResultHistory`, який забезпечує накопичення результатів кількох запусків моделювання. Після кожного збереження результату викликається метод `addResult(result: SimulationResult)`, що дозволяє сформувати залежність похибки від кількості точок.

В межах експерименту досліджується залежність точності методу Монте-Карло від кількості випадкових точок. Як змінний параметр виступає кількість точок N , а як вимірювана величина - похибка обчислення:

$$\Delta = |\pi_{\text{обчислене}} - \pi_{\text{точне}}|.$$

Отримані результати дозволяють оцінити точність методу та проаналізувати зміну похибки залежно від кількості згенерованих точок.

2.4 Висновки до розділу 2

У даному розділі виконано проектування програмного застосунку для візуалізації методу Монте-Карло. На основі визначених вимог здійснено

функціональне проектування системи, сформовано перелік основних функцій та встановлено взаємозв'язки між ними.

Побудовано діаграму варіантів використання, яка відображає основні сценарії взаємодії користувача із застосунком, а також розроблено узагальнений алгоритм роботи у вигляді UML-діаграми діяльності, що описує послідовність виконання моделювання.

Виконано проектування графічного інтерфейсу користувача, визначено структуру головного вікна, його основні елементи та сценарії взаємодії користувача із системою.

Також спроектовано структури даних та класи програмного застосунку, визначено їх атрибути, методи та взаємозв'язки, що відображено у UML-діаграмі класів. Розроблено алгоритм моделювання як метод класу MonteCarloSimulation, що забезпечує генерацію випадкових точок, накопичення статистичних даних та обчислення значення числа π .

Окрему увагу приділено організації обчислювальних експериментів: передбачено збереження результатів моделювання та формування графіку залежності похибки від кількості точок, що дозволяє оцінити точність методу Монте-Карло.

Отримані результати проектування підтверджують узгодженість між функціональними вимогами, архітектурою системи та алгоритмічним рішенням задачі, що створює основу для подальшої реалізації програмного застосунку.

3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО ЗАСТОСУНКУ ТА ДОСЛІДЖЕННЯ ТОЧНОСТІ ОЦІНКИ ЗНАЧЕННЯ ЧИСЛА π МЕТОДОМ МОНТЕ-КАРЛО

3.1 Обґрунтування вибору засобів розробки

Для реалізації програмного застосунку було обрано мову програмування Java. Вона підтримує об'єктно-орієнтований підхід, що дозволяє зручно реалізувати структуру класів і розділити логіку програми на окремі компоненти. Крім того, Java є достатньо універсальною та широко використовується для розробки прикладних програм.

Для створення графічного інтерфейсу використано бібліотеку JavaFX, що надає можливості для побудови віконного інтерфейсу, роботи з графікою та реалізації анімації. Це дозволяє відобразити процес моделювання, зокрема генерацію випадкових точок і їх класифікацію, а також реалізувати покрокове виконання алгоритму та керування швидкістю візуалізації.

У процесі розробки застосовувалась система Maven, яка використовується для організації структури проекту та його запуску. Вона спрощує роботу з програмою на етапі розробки, але не впливає на реалізацію алгоритму.

Як середовище розробки було обрано IntelliJ IDEA, яке забезпечує зручну роботу з кодом, підтримку Maven та засоби для налагодження програми.

Отже, обрані засоби розробки дозволяють реалізувати програмний застосунок відповідно до поставлених вимог та забезпечують необхідні можливості для візуалізації алгоритму методу Монте-Карло.

3.2 Розробка структури програмного застосунку

Структура програмного застосунку сформована на основі результатів проектування, наведених у розділі 2, і реалізована у вигляді окремих взаємодіючих компонентів.

Програму поділено на три групи:

- компоненти інтерфейсу користувача - MainApp;
- компоненти логіки керування - методи startSimulation(), pauseSimulation(), resetSimulation(), saveResult() у MainApp;
- компоненти предметної області та алгоритму - MonteCarloSimulation, IMonteCarloAlgorithm, Point, Statistics, SimulationParameters, IterationState, SimulationResult, ResultHistory, VisualizationMode.

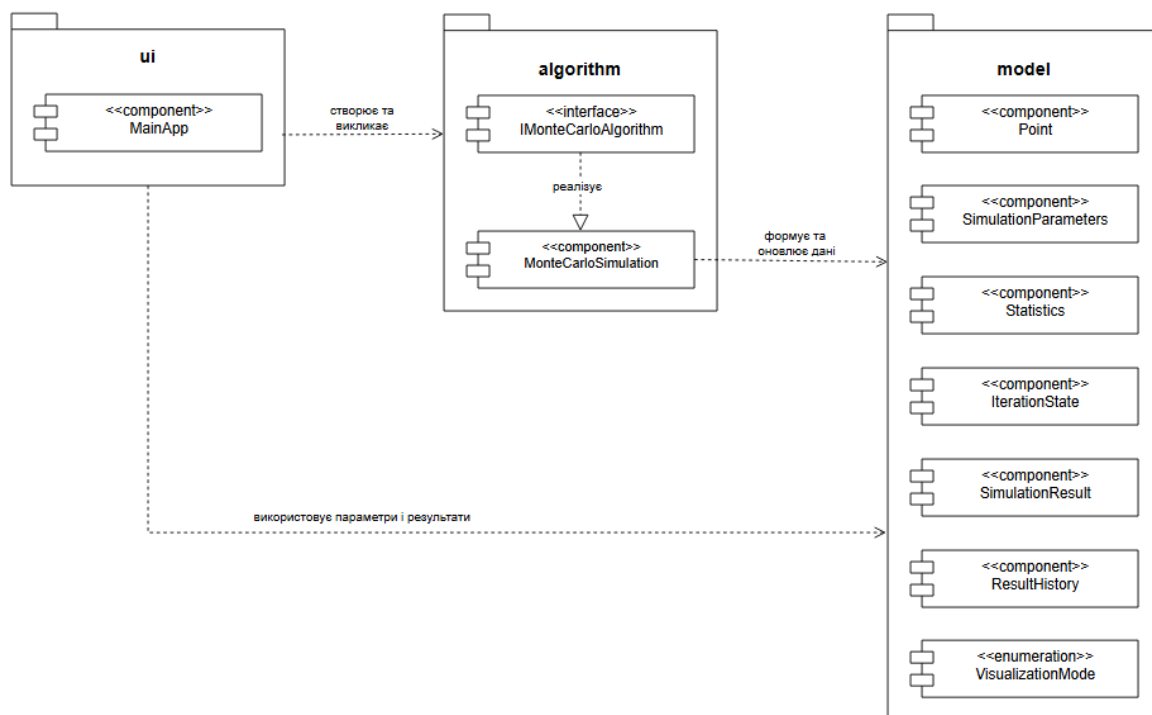


Рисунок 3.1 - Структура програмного застосунку у вигляді UML-діаграми компонентів

На рисунку 3.1 показано основні компоненти системи та зв'язки між ними.

До інтерфейсу користувача належить компонент MainApp. Він відповідає за роботу з користувачем: отримує введені параметри, запускає моделювання та відображає результати.

Алгоритмічна частина складається з інтерфейсу IMonteCarloAlgorithm і його реалізації MonteCarloSimulation. Інтерфейс задає набір методів для виконання моделювання (ініціалізація, покрокове виконання, перевірка завершення та формування результату), а клас MonteCarloSimulation реалізує ці дії та безпосередньо виконує алгоритм Монте-Карло.

До компонентів даних належать класи: Point, SimulationParameters, Statistics, IterationState, SimulationResult, ResultHistory, а також перелічуваний тип VisualizationMode. Вони використовуються для зберігання параметрів моделювання, проміжних станів і результатів.

Компонент MainApp використовує методи, визначені інтерфейсом IMonteCarloAlgorithm, реалізацію яких забезпечує клас MonteCarloSimulation. У свою чергу, MonteCarloSimulation працює з класами даних: створює точки, оновлює статистику та формує результати.

Після завершення обчислень компонент MainApp отримує результат виконання алгоритму через виклики методів інтерфейсу IMonteCarloAlgorithm та використовує ці дані для відображення користувачу.

Така структура дозволяє розділити інтерфейс, алгоритм і дані. Алгоритм працює незалежно від графічного інтерфейсу, а інтерфейс лише використовує готові результати для відображення.

3.3 Реалізація алгоритму та програмного застосунку

Реалізація програмного застосунку виконана відповідно до спроектованої у розділі 2 структури та базується на об'єктно-орієнтованому підході. Основна логіка методу Монте-Карло реалізована у класі MonteCarloSimulation, який забезпечує виконання моделювання.

Клас MonteCarloSimulation реалізує інтерфейс IMonteCarloAlgorithm, що визначає основні методи роботи алгоритму, зокрема ініціалізацію параметрів та покрокове виконання моделювання. Перед початком роботи у компоненті MainApp створюється об'єкт SimulationParameters, після чого викликається метод initialize(parameters), у якому встановлюються параметри моделювання та скидаються статистичні дані.

У процесі виконання алгоритму використовується метод nextStep(), який відповідає за одну ітерацію моделювання. Під час виконання цього методу створюється об'єкт Point із випадковими координатами. Далі викликається метод

update(point) класу Statistics, у межах якого здійснюється перевірка належності точки до кола за допомогою виклику isInsideCircle(). Після цього формується об'єкт IterationState, який містить інформацію про поточний стан моделювання та повертається до компонента MainApp.

Для виконання моделювання у повному обсязі метод nextStep() викликається багаторазово до досягнення заданої кількості точок, що відображено на діаграмі у вигляді циклу.

Програмний застосунок підтримує анімований режим виконання, у якому ітерації виконуються послідовно, а після кожної ітерації відбувається оновлення відображення точки та статистичних даних.

Взаємодія між компонентами реалізується шляхом виклику методів. Компонент MainApp ініціює виконання алгоритму, створює параметри моделювання, викликає методи алгоритму та використовує отримані результати для оновлення графічного інтерфейсу.

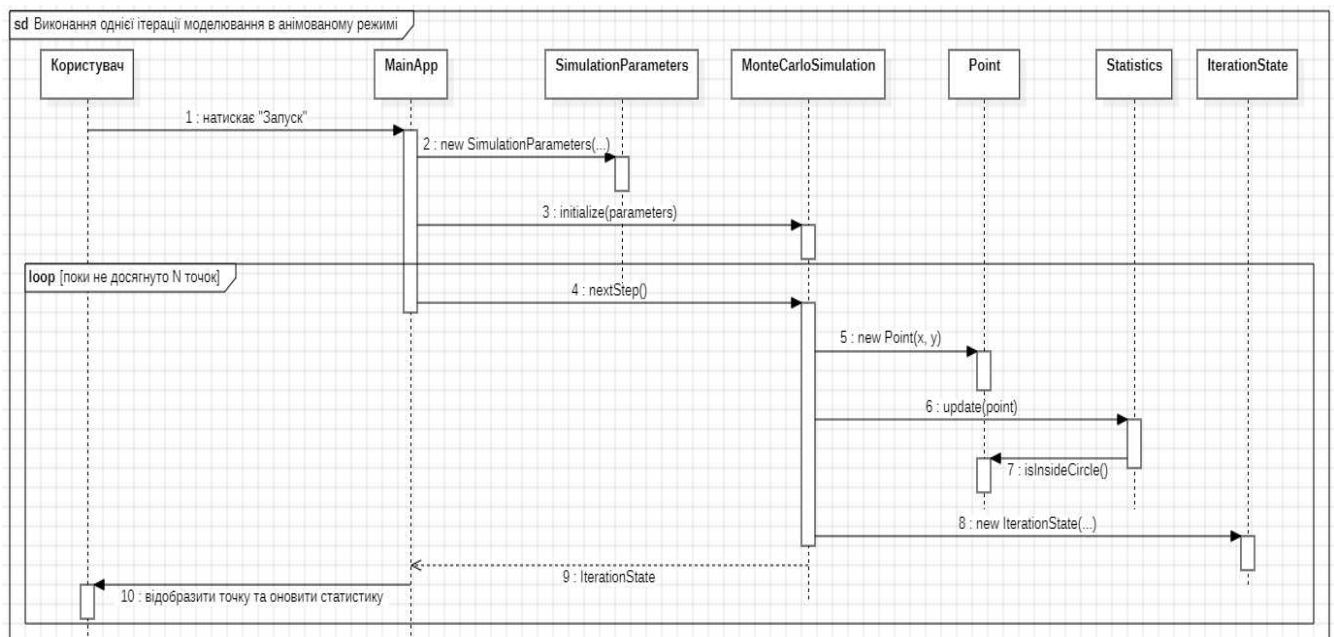


Рисунок 3.2 - UML-діаграма послідовності виконання алгоритму

На рисунку 3.2 наведено UML-діаграму послідовності, яка відображає взаємодію компонентів під час виконання однієї ітерації моделювання в анімованому режимі. На діаграмі показано ланцюжок викликів методів між об'єктами, що забезпечує реалізацію алгоритму.

Користувач ініціює запуск моделювання, після чого у компоненті MainApp створюється об'єкт SimulationParameters та викликається метод initialize(parameters). Далі у циклі виконується метод nextStep(), результатом якого є об'єкт IterationState, що повертається до компонента MainApp і використовується для оновлення відображення точки та статистичних даних.

Таким чином, реалізація алгоритму відповідає спроектованій структурі та забезпечує виконання моделювання з можливістю покрокового відображення результатів.

Фрагменти програмного коду, що відповідають наведеній UML-діаграмі послідовності, наведені у додатку А.

Інструкція користувача щодо роботи з програмним застосунком наведена у додатку Б.

Повні тексти програмних компонентів, розроблених у межах курсової роботи, наведено у додатку В.

3.4 Тестування програмного застосунку

Тестування програмного застосунку виконано з метою перевірки відповідності реалізованих функцій вимогам, сформульованим у розділі 1, та функціональній моделі, наведеної у розділі 2. Основну увагу приділено перевірці введення параметрів моделювання, запуску алгоритму, візуалізації випадкових точок, обчисленню значення числа π , керуванню анімованим режимом, скиданню та збереженню результатів.

Для перевірки працездатності програми було розроблено тест-кейси для основних функцій другого рівня ієрархії, визначених у підрозділі 2.1.

Результати тестування наведено в таблиці 3.1.

Таблиця 3.1 - Тест-кейси програмного застосунку для візуалізації методу Монте-Карло

№	Функція	Вхідні дані / дія	Очікуваний результат	Фактичний результат
1	Ф1.1 Введення кількості точок	$N = 1000$	Значення приймається, моделювання може бути запущене	Значення прийнято, моделювання запускається
2	Ф1.1 Введення кількості точок	$N = abc$	Програма не запускає моделювання і виводить повідомлення про помилку	Виведено повідомлення про те, що кількість точок має бути цілим числом
3	Ф1.1 Введення кількості точок	$N = 50$	Програма не запускає моделювання і повідомляє про обмеження діапазону	Виведено повідомлення про те, що кількість точок має бути від 100 до 100000
4	Ф1.1 Введення кількості точок	$N = 150000$	Програма не запускає моделювання і повідомляє про обмеження діапазону	Виведено повідомлення про допустимий діапазон кількості точок
5	Ф5.2 Вибір режиму візуалізації	Обрано автоматичний режим, $N = 1000$	Моделювання виконується до завершення без паузи та покрокового керування	Усі точки згенеровано, результати відображено
6	Ф5.2, Ф5.3 Вибір анімованого режиму та керування анімацією	Обрано анімований режим, $N = 1000$	Точки відображаються поступово, доступна пауза	Точки відображаються поступово, кнопка «Пауза» зупиняє виконання
7	Ф5.3 Керування анімованою візуалізацією	Натиснуто «Пауза», потім «Продовжити»	Після паузи моделювання зупиняється, після продовження - виконується далі	Анімацію призупинено та успішно продовжено
8	Ф1.2 Встановлення швидкості анімації	Встановлено мінімальне значення швидкості	Анімація відбувається повільно	Точки з'являються повільно

Продовження таблиці 3.1

№	Функція	Вхідні дані / дія	Очікуваний результат	Фактичний результат
9	Ф1.2 Встановлення швидкості анімації	Встановлено максимальне значення швидкості	Анімація відбувається швидко	Точки з'являються швидко
10	Ф5.4 Покрокове виконання алгоритму	Після паузи натиснуто «Крок»	Виконується одна наступна ітерація алгоритму	Додано одну точку, статистичні дані оновлено
11	Ф3.1-Ф3.3 Візуалізація процесу моделювання	Запуск моделювання	Відображається квадрат, коло та точки різних кольорів залежно від належності до кола	Геометрична область і класифіковані точки відображаються коректно
12	Ф4.1-Ф4.2 Обчислення та відображення результатів	Завершене моделювання	Відображаються значення π , кількість точок, кількість точок у колі та похибка	Результати обчислення відображено у відповідній області інтерфейсу
13	Ф6.1, Ф6.3 Збереження результатів та оновлення графіка	Після завершення натиснуто «Зберегти результат»	Результат додається до історії, графік похибки оновлюється	На графіку додано нову точку залежності похибки від кількості точок
14	Ф5.5 Скидання результатів	Натиснуто «Скидання»	Графічна область очищується, статистичні дані обнулюються	Поле моделювання очищено, показники скинуто

Під час тестування було перевірено коректність роботи основних функцій програмного застосунку, зокрема введення параметрів моделювання, вибір режиму візуалізації, керування анімованим виконанням алгоритму, оновлення статистичних даних та відображення результатів обчислення. Також перевірено обробку некоректного введення даних та роботу функцій збереження і скидання результатів. Результати тестування підтвердили коректність реалізації основних функцій програмного застосунку.

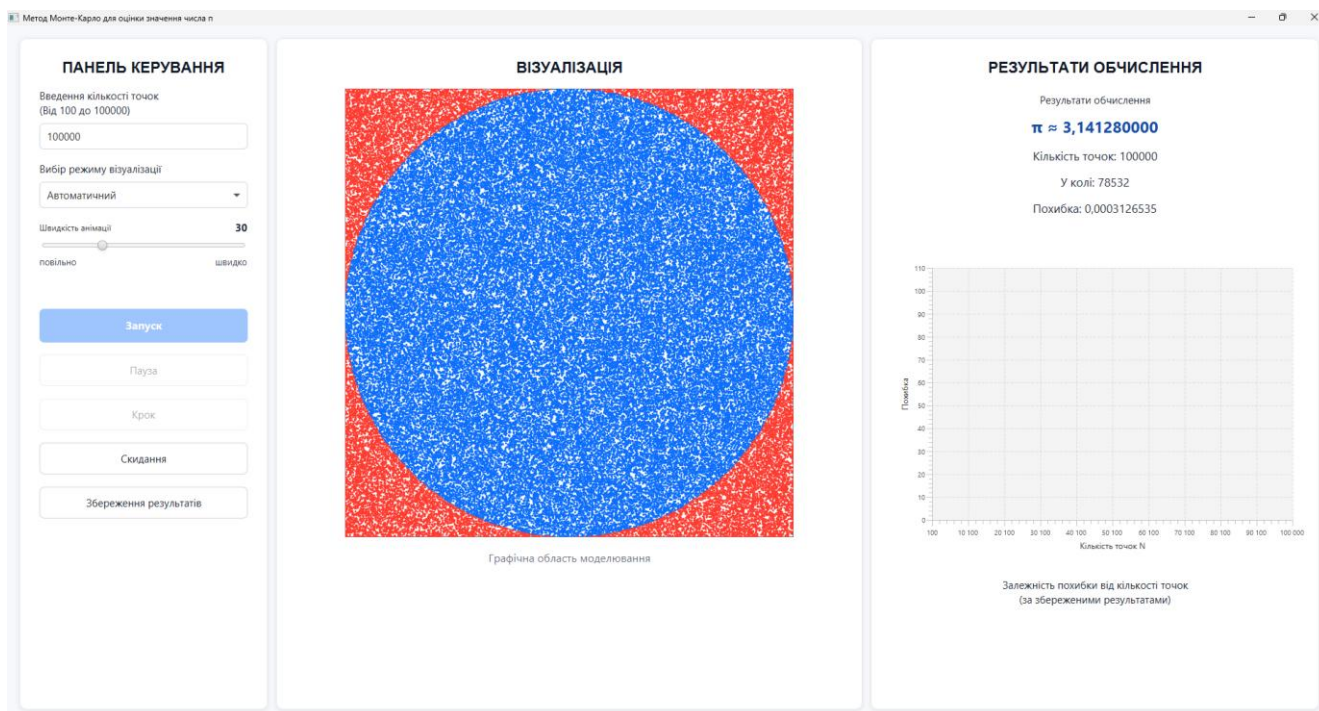


Рисунок 3.3 - Результат виконання моделювання в автоматичному режимі

На рисунку 3.3 наведено результат виконання моделювання в автоматичному режимі. Даний результат підтверджує коректність виконання тест-кейсів 1, 5, 11 та 12, пов'язаних із введенням допустимої кількості точок, запуском моделювання, візуалізацією процесу та відображенням результатів обчислення.

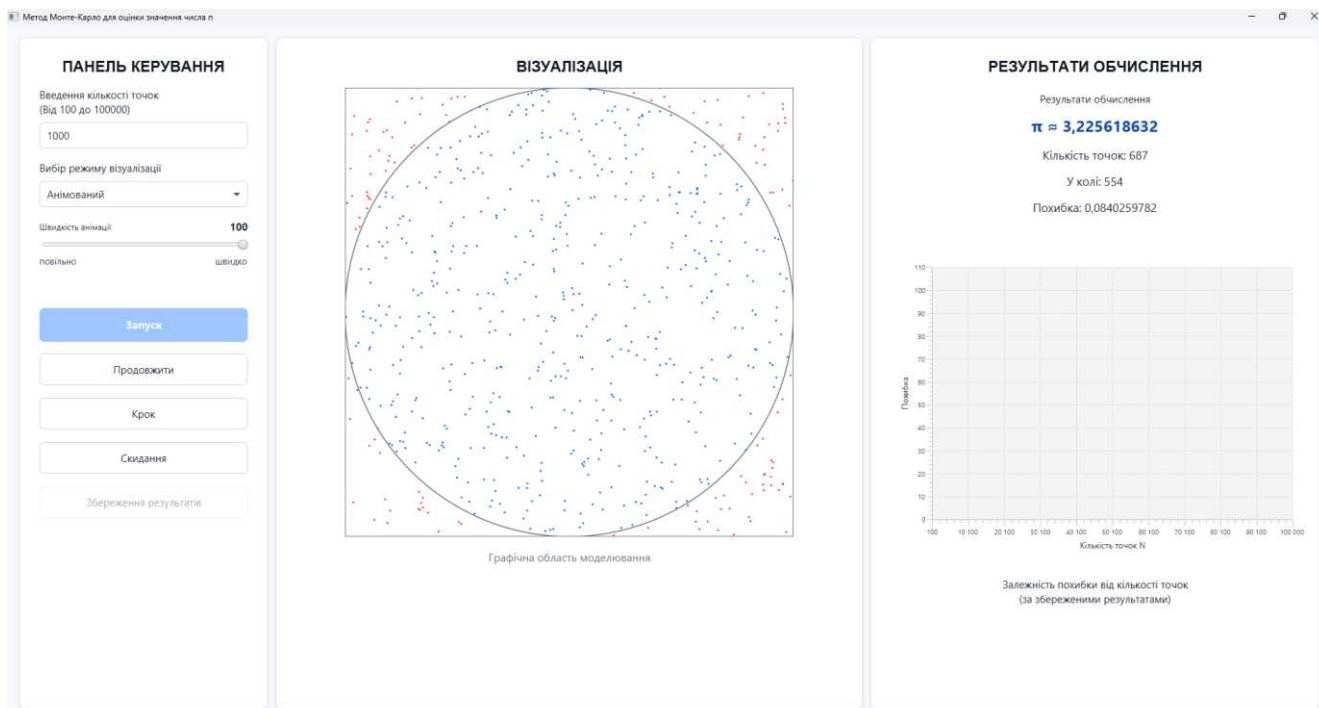


Рисунок 3.4 - Виконання моделювання в анімованому режимі після призупинення

На рисунку 3.4 показано роботу програмного застосунку в анімованому режимі після призупинення моделювання. Даний сценарій підтверджує виконання тест-кейсів 6 і 7, що пов'язані з анімованим режимом та призупиненням моделювання.

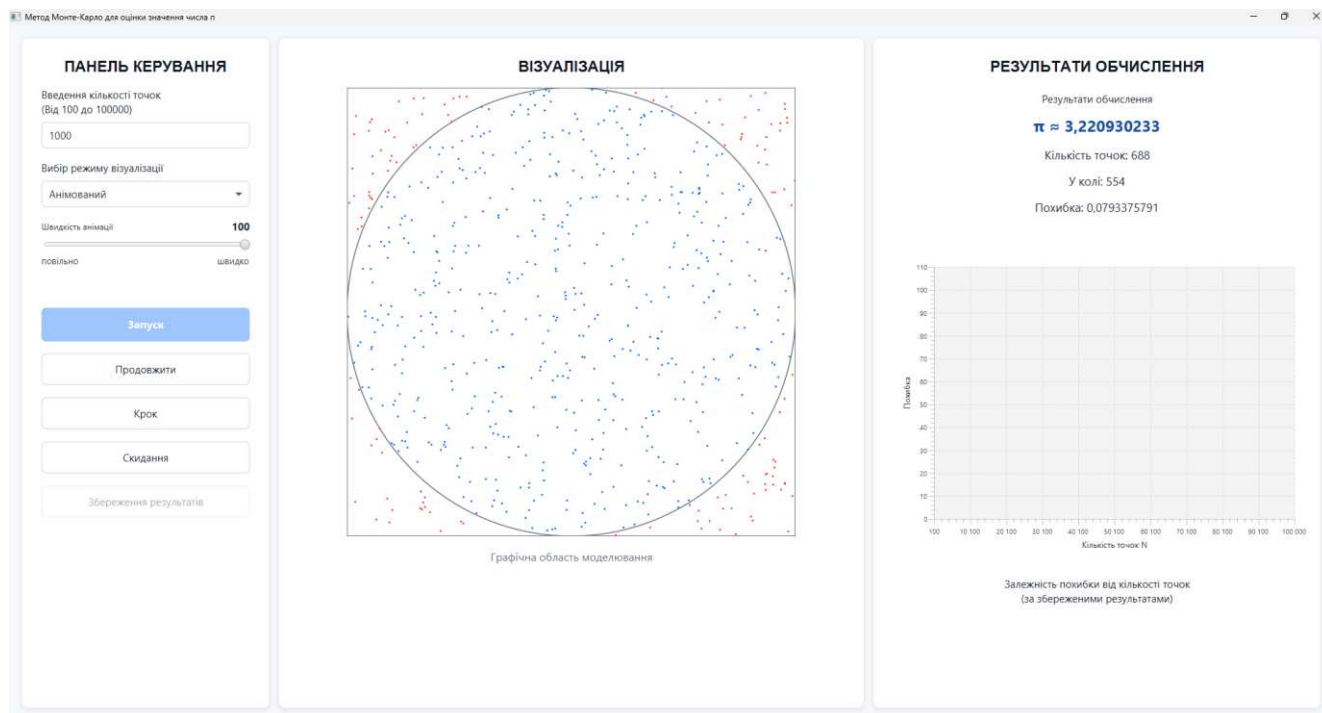


Рисунок 3.5 - Покрокове виконання алгоритму після призупинення анімованого режиму

На рисунку 3.5 наведено приклад покрокового виконання алгоритму після призупинення анімованого режиму. Даний результат підтверджує коректність виконання тест-кейсу 10, пов'язаного з виконанням однієї ітерації алгоритму за допомогою кнопки «Крок».

На рисунку 3.6 наведено приклад повідомлення про помилку при введенні некоректного значення кількості точок. Даний результат підтверджує виконання тест-кейсів 2-4, пов'язаних із перевіркою коректності введених даних.

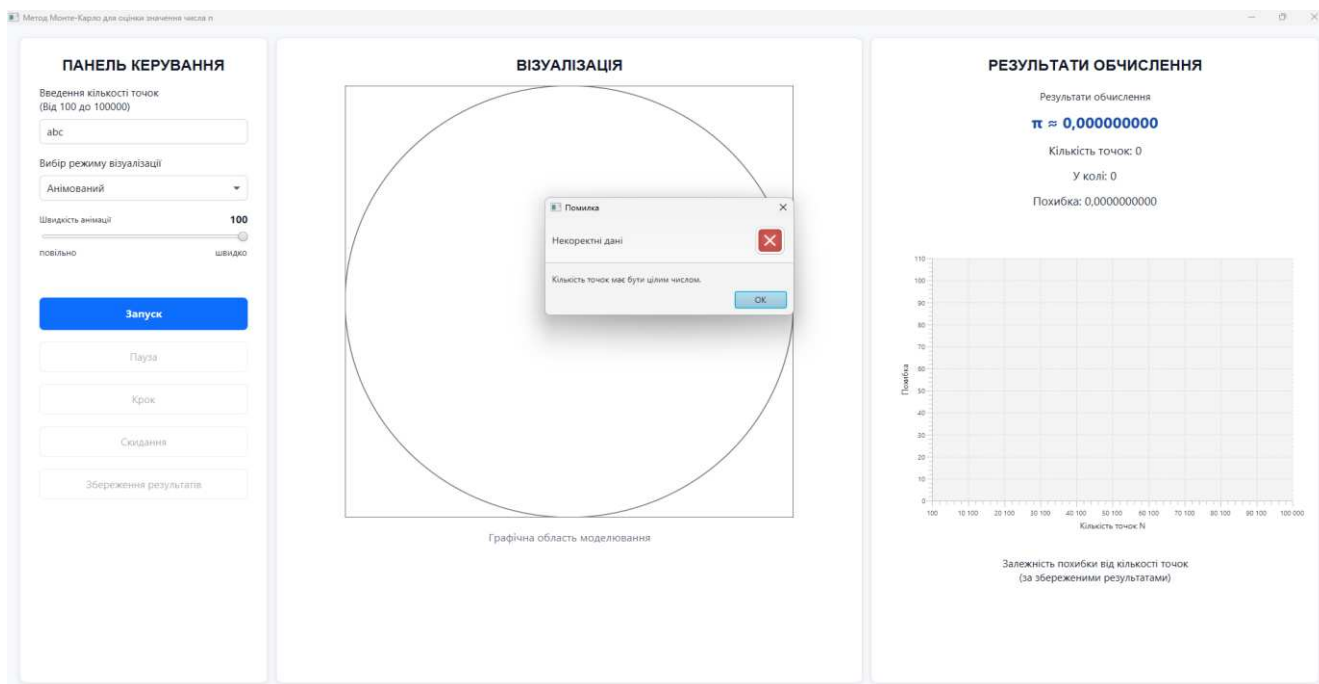


Рисунок 3.6 - Повідомлення про помилку при некоректному введенні кількості точок

На рисунку 3.7 показано оновлення графіка похибки після збереження результатів моделювання. Даний результат підтверджує коректність виконання тест-кейсу 13, пов'язаного зі збереженням результатів та оновленням графіка залежності похибки від кількості введених точок.

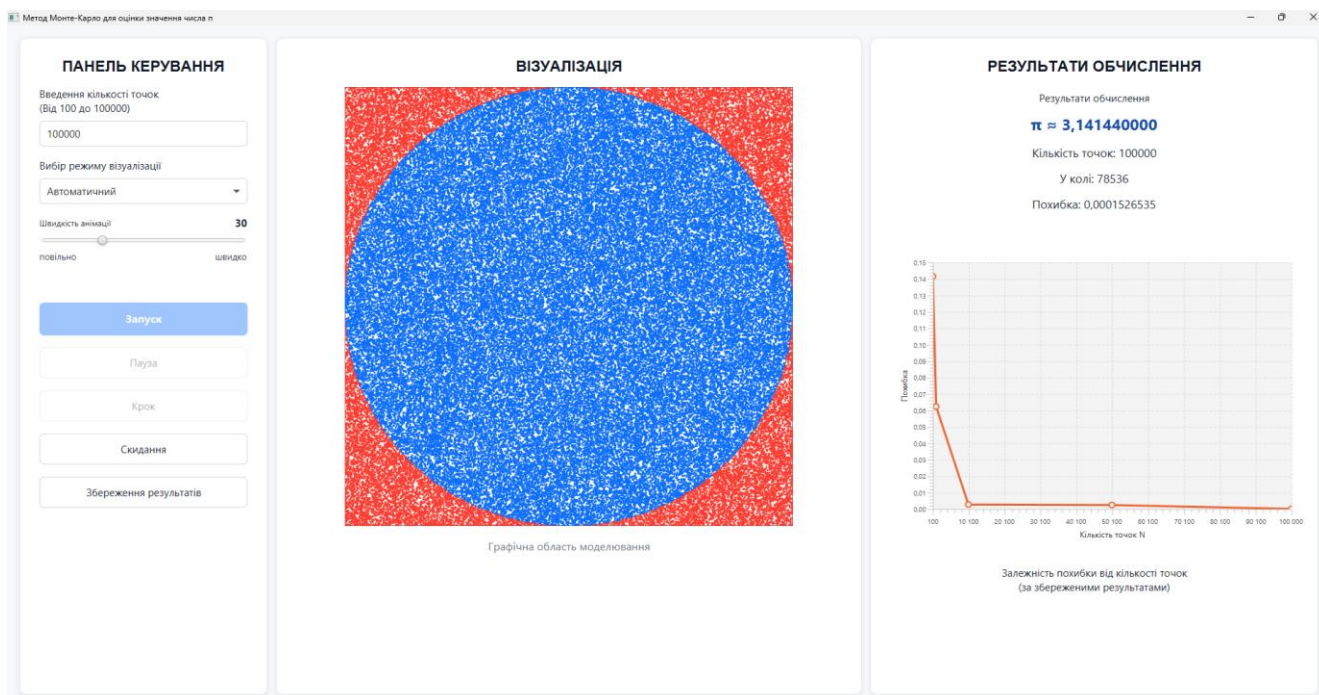


Рисунок 3.7 - Оновлення графіка похибки після збереження результатів моделювання

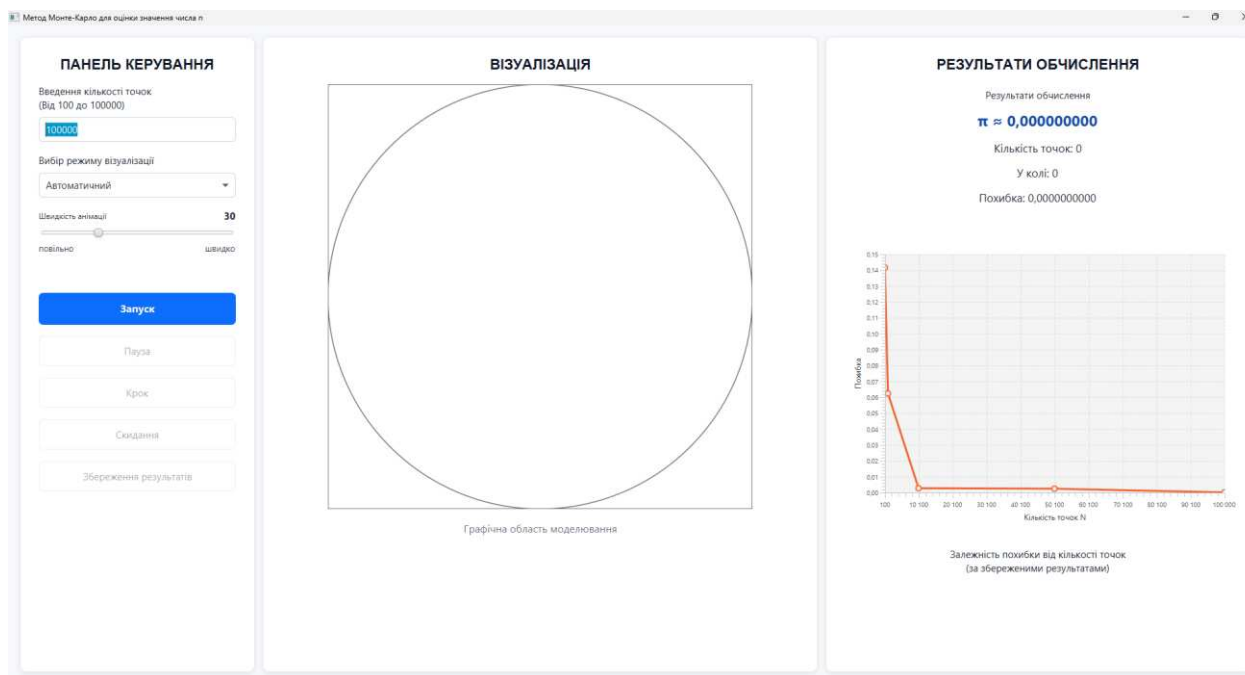


Рисунок 3.8 - Стан програми після скидання результатів

На рисунку 3.8 наведено стан програмного застосунку після скидання результатів моделювання. Даний результат підтверджує виконання тест-кейсу 14, пов'язаного зі скиданням результатів та очищенням графічної області моделювання. При цьому раніше збережені результати залишаються на графіку залежності похибки, що дозволяє виконувати подальший порівняльний аналіз результатів моделювання.

3.5 Експериментальні дослідження точності методу

Для оцінки точності методу Монте-Карло було проведено серію обчислювальних експериментів. Метою дослідження є визначення залежності похибки обчислення числа π від кількості згенерованих випадкових точок. У процесі експерименту змінювалась кількість точок N , а для кожного запуску фіксувалось отримане значення числа π та абсолютна похибка Δ .

У межах дослідження використовувались такі значення кількості точок:

100, 500, 1000, 5000, 10000, 20000, 50000, 75000, 90000, 100000.

Для кожного значення виконувалось окреме моделювання методом Монте-Карло, після чого фіксувались:

- наближене значення числа π за формулою:

$$\pi \approx 4 \cdot \left(\frac{k}{N}\right),$$

де k - кількість точок у колі, N - загальна кількість точок;

- абсолютна похибка обчислення за формулою:

$$\Delta = |\pi_{\text{обчислене}} - \pi_{\text{точне}}|.$$

Результати проведених експериментів наведено в таблиці 3.2.

Таблиця 3.2 Результати експериментального дослідження точності методу Монте-Карло для обчислення значення числа π

№ експерименту	Кількість точок N	Обчислене π	Абсолютна похибка Δ
1	100	3,240000000	0,0984073465
2	500	3,208000000	0,0664073465
3	1000	3,176000000	0,0344073465
4	5000	3,167200000	0,0256073465
5	10000	3,134000000	0,0075926535
6	20000	3,148200000	0,0066073465
7	50000	3,146240000	0,0046473465
8	75000	3,140160000	0,0014326535
9	90000	3,140933333	0,0006593202
10	100000	3,141720000	0,0001273465

На основі отриманих результатів побудовано графік залежності похибки обчислення від кількості згенерованих точок.

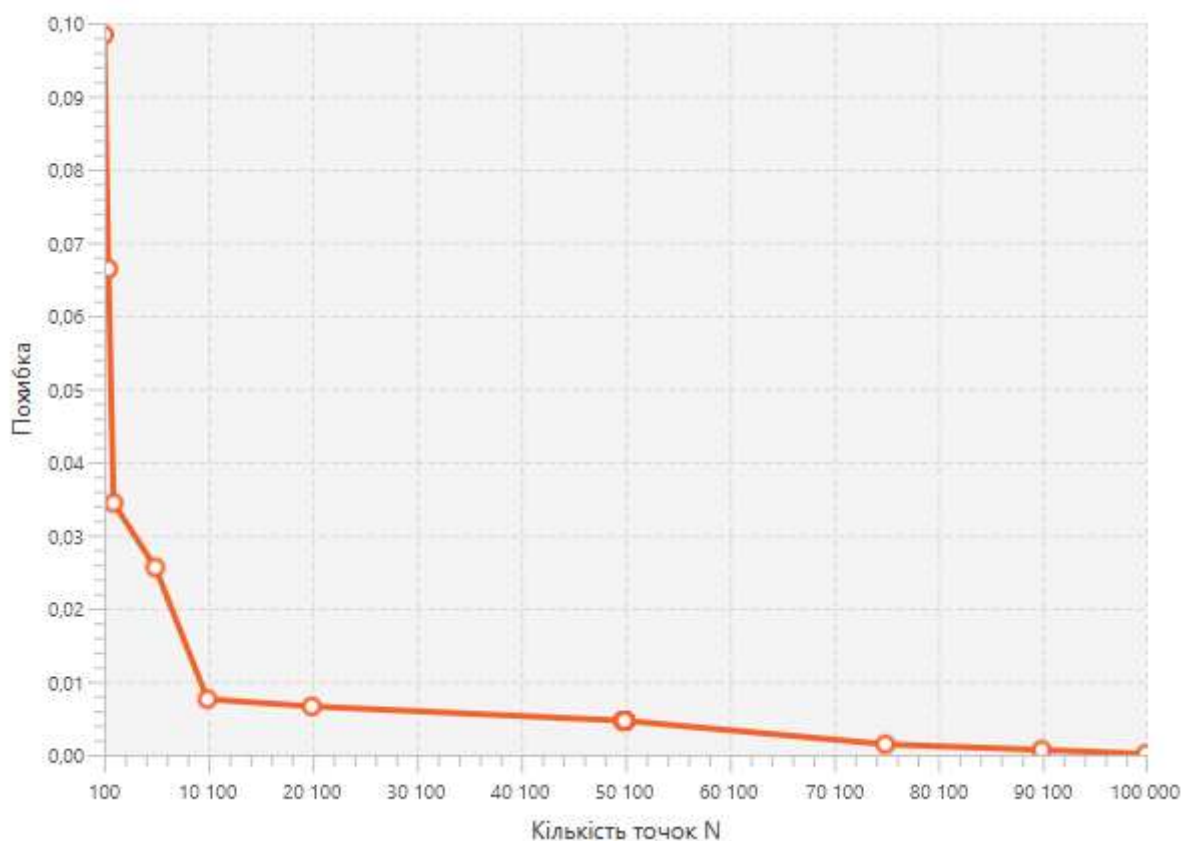


Рисунок 3.9 - Залежність похибки обчислення значення числа π від кількості згенерованих точок

Отримані результати показують, що зі збільшенням кількості випадкових точок похибка обчислення числа π загалом зменшується.

Для малих значень кількості точок N спостерігаються значні відхилення результату, що пояснюється статистичною природою методу Монте-Карло та недостатньою кількістю випадкових випробувань.

Зі збільшенням кількості згенерованих точок розподіл випадкових точок стає більш рівномірним, а обчислене значення числа π стає більш точним.

Також варто зазначити, що метод Монте-Карло є стохастичним, тому похибка не завжди зменшується строго монотонно для кожного окремого запуску. Проте загальна тенденція результатів підтверджує підвищення точності при збільшенні кількості згенерованих точок.

Найменша похибка була отримана при $N=100000$, а саме $\Delta=0,0001273465$ для обчисленого значення $\pi=3,141720000$.

3.6 Висновки до розділу 3

У даному розділі було реалізовано програмний застосунок для графічної інтерпретації методу Монте-Карло для оцінки значення числа π . Реалізацію виконано відповідно до спроектованої архітектури та функціональних вимог, визначених у розділах 1,2.

Було реалізовано генерацію випадкових точок, обчислення наближеного значення числа π , накопичення статистичних даних, а також автоматичний та анімований режими візуалізації процесу моделювання. Крім цього, реалізовано засоби керування виконанням алгоритму, покрокове виконання, збереження результатів та побудову графіка залежності похибки від кількості згенерованих точок.

Під час тестування підтверджено коректність роботи основних функцій програмного застосунку, зокрема введення параметрів, запуску моделювання, відображення результатів та обробки помилок введення даних.

В межах експериментального дослідження проаналізовано залежність точності оцінки числа π від кількості згенерованих точок. Отримані результати підтвердили, що зі збільшенням кількості точок похибка обчислення загалом зменшується, а значення числа π стає більш точним.

Отже, поставлену мету курсової роботи досягнуто, а реалізований програмний застосунок забезпечує наочну візуалізацію та дослідження точності методу Монте-Карло для оцінювання значення числа π .

ВИСНОВКИ

У ході виконання курсової роботи було досліджено метод Монте-Карло та особливості його використання для оцінювання значення числа π . Проведений аналіз показав, що даний метод базується на статистичному моделюванні та дозволяє отримувати наближені результати за допомогою генерації випадкових величин.

В межах роботи було розглянуто математичну модель оцінювання числа π методом Монте-Карло, сформовано вимоги до програмного застосунку та спроектовано його архітектуру. Для опису структури та функціональності системи було побудовано UML-діаграми варіантів використання, діяльності, класів, компонентів і послідовності.

У результаті виконання роботи розроблено програмний застосунок мовою Java з використанням бібліотеки JavaFX, який забезпечує графічну візуалізацію процесу моделювання методом Монте-Карло. Програма підтримує автоматичний та анімований режими роботи, покрокове виконання алгоритму, відображення статистичних даних, а також побудову графіка залежності похибки від кількості згенерованих точок.

Під час тестування було перевірено коректність роботи основних функцій застосунку, зокрема введення параметрів моделювання, генерації випадкових точок, обчислення значення числа π , роботи режимів візуалізації та обробки некоректних даних. Результати тестування підтвердили працездатність програмного застосунку та відповідність реалізованих функцій поставленим вимогам.

При експериментальному дослідженні було проаналізовано залежність точності оцінки числа π від кількості згенерованих точок. Отримані результати підтвердили, що зі збільшенням кількості точок похибка обчислення загалом зменшується, а наближене значення π стає ближчим до точного.

Перевагою розробленого застосунку є наочна візуалізація процесу роботи методу Монте-Карло та можливість дослідження впливу кількості точок на

точність результату. До обмежень можна віднести залежність точності від випадкового характеру генерації точок та необхідність великої кількості ітерацій для отримання більш точних результатів.

Отже, розроблений програмний застосунок може використовуватись як навчальний інструмент для демонстрації принципу роботи методу Монте-Карло та основ статистичного моделювання. Подальше вдосконалення програми може бути пов'язане з розширенням набору задач, реалізованих методом Монте-Карло, а також із додаванням нових засобів статистичного аналізу результатів моделювання.

ДЖЕРЕЛА ПОСИЛАННЯ

1. Моделювання випадкових величин Монте-Карло [Електронний ресурс] : [Веб-сайт]. - Електронні дані. - Режим доступу: <https://caseware.com.ua/monte-carlo-3/> (дата звернення 12.03.2026) - Назва з екрана.
2. Metropolis N, Ulam S. The Monte Carlo method: стаття/ J. Amer. Statistical assoc. 1949. Vol. 44. №247.; [самостійний переклад з англійської]; 1949. - с.335-341.
3. Лисиченко Г.В., Забулонов Ю.Л., Хміль Г.А. Природний, техногенний та екологічний ризики : аналіз, оцінка, управління - [2008-22924] УДК 005.334:502.1 ББК 20.1 Л63- Наук. Думка - 2008 - 541 с.
4. D.P. Kroese, T. Taimre, Z.I. Botev. Monte Carlo Methods : рукопис / Dirk P. Kroese - Department of Mathematics School of Mathematics and Physics The University of Queensland; [самостійний переклад з англійської]; 2011. - 176 с.
5. Матвіїшина Н.В., Кондрат'єва Н.О. Інформаційне забезпечення статистичних досліджень : методичні рекомендації / Матвіїшина Н.В., Кондрат'єва Н.О. - Запоріжжя , 2020. - 51 с. - УДК: 311.21:004(076.5) М338.
6. Крамаренко Т. Г. Використання методу Монте-Карло у навчанні стохастики в контексті підготовки учителів математики до впровадження STEM-освіти : стаття наукового журналу Фізико-математична освіта/ Крамаренко Т.Г. - 2023 - DOI 10.31110/2413-1571-2023-038-4-006.
7. Монте-Карло симуляція [Електронний ресурс] : [Веб-сайт]. - Електронні дані. - Режим доступу: <https://www.vpnunlimited.com/ua/help/cybersecurity/monte-carlo-simulation?srsId=AfmBOooBn6-SO4NjcQ246aqbIyeK37m3CbPPVGnwkmIKd9D3h-66IbKn> (дата звернення 13.03.2026) - Назва з екрана.
8. Метод Монте-Карло для Обчислення Інтегралів: Основи та Застосування [Електронний ресурс] : [Веб-сайт]. - Електронні дані. - Режим доступу: <https://www.mathros.net.ua/chyselne-integruvannja-metodom-monte-karlo.html> (дата звернення 13.03.2026) - Назва з екрана.

ДОДАТОК А

Фрагменти програмного коду, що відповідають UML-діаграмі послідовності виконання алгоритму

Лістинг А.1 - Фрагмент запуску моделювання та виконання анімованого режиму в класі MainApp

Файл: src/main/java/com/coursework/montecarlo/ui/MainApp.java

```
private void startSimulation() {
    try {
        int totalPoints = Integer.parseInt(pointsField.getText().trim());
        SimulationParameters params = new SimulationParameters(totalPoints, (int)
speedSlider.getValue(), modeBox.getValue());

        simulation.initialize(params);
        currentResult = null;
        pauseButton.setText("Пауза");
        drawBaseArea();
        updateLabels(0, 0, 0, 0);

        if (params.getMode() == VisualizationMode.AUTOMATIC) {
            runAutomaticMode(totalPoints);
            setFinishedButtonState();
        } else {
            startAnimatedMode(params);
            setAnimatedRunningButtonState();
        }
    } catch (NumberFormatException ex) {
        showError("Кількість точок має бути цілим числом.");
    } catch (IllegalArgumentException ex) {
        showError(ex.getMessage());
    }
}

private void startAnimatedMode(SimulationParameters params) {
    if (timeline != null) {
        timeline.stop();
    }
    double delay = Math.max(3, 900.0 / params.getAnimationSpeed());
    timeline = new Timeline(new KeyFrame(Duration.millis(delay), e ->
executeOneStep()));
    timeline.setCycleCount(Timeline.INDEFINITE);
}
```



```

        timeline.play();
    }

    private void executeOneStep() {
        try {
            IterationState state = simulation.nextStep();
            if (state.getPoint() != null) {
                drawPoint(state.getPoint());
            }
            updateLabels(state.getGeneratedPoints(), state.getInsideCirclePoints(),
state.getPiEstimate(), state.getError());
            if (state.isFinished()) {
                if (timeline != null) {
                    timeline.stop();
                }
                currentResult = simulation.createResult();
                updateLabels(currentResult.getTotalPoints(),
currentResult.getInsideCirclePoints(), currentResult.getPiEstimate(),
currentResult.getError());
                setFinishedButtonState();
            }
        } catch (IllegalStateException ex) {
            showError("Спочатку натисніть кнопку 'Запуск'.");
        }
    }
}

```

ЛІСТИНГ А.2 - Реалізація однієї ітерації в класі MonteCarloSimulation

Файл:

src/main/java/com/coursework/montecarlo/algorithm/MonteCarloSimulation.java

```

public void initialize(SimulationParameters parameters) {
    this.parameters = parameters;
    this.statistics.reset();
    this.startTime = System.currentTimeMillis();
}

public IterationState nextStep() {
    ensureInitialized();

    if (isFinished()) {
        return createState(null, true);
    }
}

```

```

    }

    Point point = generatePoint();
    statistics.update(point);
    return createState(point, isFinished());
}

private Point generatePoint() {
    return new Point(random.nextDouble(), random.nextDouble());
}

private IterationState createState(Point point, boolean finished) {
    return new IterationState(
        point,
        statistics.getGeneratedPoints(),
        statistics.getInsideCirclePoints(),
        statistics.calculatePi(),
        statistics.calculateError(),
        finished
    );
}

public boolean isFinished() {
    return parameters != null && statistics.getGeneratedPoints() >=
parameters.getTotalPoints();
}

```

Лістинг А.3 - Оновлення статистики в класі Statistics

Файл: src/main/java/com/coursework/montecarlo/model/Statistics.java

```

public void update(Point point) {
    generatedPoints++;
    if (point.isInsideCircle()) {
        insideCirclePoints++;
    }
}

```

Лістинг А.4 - Перевірка належності точки до кола в класі Point

Файл: src/main/java/com/coursework/montecarlo/model/Point.java

```

public boolean isInsideCircle() {
    double dx = x - 0.5;
    double dy = y - 0.5;
    return dx * dx + dy * dy <= 0.25;
}

```

Лістинг А.5 - Структура стану ітерації в класі IterationState

Файл: src/main/java/com/coursework/montecarlo/model/IterationState.java

```

private final Point point;
private final int generatedPoints;
private final int insideCirclePoints;
private final double piEstimate;
private final double error;
private final boolean finished;

public IterationState(Point point, int generatedPoints, int insideCirclePoints,
    double piEstimate, double error, boolean finished) {
    this.point = point;
    this.generatedPoints = generatedPoints;
    this.insideCirclePoints = insideCirclePoints;
    this.piEstimate = piEstimate;
    this.error = error;
    this.finished = finished;
}

public Point getPoint() { return point; }
public int getGeneratedPoints() { return generatedPoints; }
public int getInsideCirclePoints() { return insideCirclePoints; }
public double getPiEstimate() { return piEstimate; }
public double getError() { return error; }
public boolean isFinished() { return finished; }

```

ДОДАТОК Б

Інструкція користувача щодо роботи з програмним застосунком

Для коректної роботи програмного застосунку необхідне таке програмне та апаратне середовище:

- операційна система Windows 10 або Windows 11;
- Java Development Kit версії 17 або вище;
- система збирання Maven;
- середовище розробки IntelliJ IDEA або інше середовище, що підтримує Maven-проекти;
- оперативна пам'ять не менше 4 ГБ;
- екран з роздільною здатністю, достатньою для відображення головного вікна програми.

Для запуску програмного застосунку необхідно відкрити проект у середовищі IntelliJ IDEA. Після відкриття проекту слід дочекатися завантаження Maven-залежностей. Далі у правій частині середовища необхідно відкрити панель Maven, перейти до пункту Plugins, відкрити групу javafx та запустити команду `javafx:run`. Після виконання цієї команди відкривається головне вікно програмного застосунку.

Після запуску програми користувач вводить кількість випадкових точок у відповідне поле. Значення має бути цілим числом у межах від 100 до 100000. Якщо введено некоректне значення, програма не запускає моделювання та виводить повідомлення про помилку.

Далі користувач обирає режим візуалізації. В автоматичному режимі всі точки генеруються одразу, після чого на екрані відображаються підсумкові результати моделювання. В анімованому режимі точки відображаються поступово, що дозволяє спостерігати процес виконання алгоритму.

У разі вибору анімованого режиму користувач може встановити швидкість візуалізації за допомогою відповідного повзунка. Мінімальне значення швидкості забезпечує повільне відображення точок, а максимальне - швидке виконання анімації.

Для початку моделювання необхідно натиснути кнопку «Запуск». Після цього програма ініціалізує параметри моделювання, очищує графічну область та починає генерацію випадкових точок. У графічній області відображається квадрат із вписаним колом. Точки, що потрапили до кола, та точки, що не потрапили до кола, позначаються різними кольорами.

В анімованому режимі користувач може призупинити виконання моделювання за допомогою кнопки «Пауза». Після призупинення доступне продовження виконання, скидання результатів або покрокове виконання алгоритму. Кнопка «Крок» виконує одну наступну ітерацію моделювання, тобто генерує одну нову точку, відображає її в графічній області та оновлює статистичні дані.

Після завершення моделювання в області результатів відображається остаточне наближене значення числа π , загальна кількість згенерованих точок, кількість точок, що потрапили до кола, та абсолютна похибка обчислення.

Для збереження результату користувач натискає кнопку «Збереження». Після цього результат поточного запуску у вигляді нової точки додається до графіку залежності похибки від кількості точок. Це дозволяє порівнювати результати декількох запусків моделювання.

Для очищення поточного моделювання користувач може натиснути кнопку «Скидання». Після цього графічна область очищується, статистичні показники обнуляються, а програма повертається до початкового стану для проведення нового експерименту. Раніше збережені результати на графіку залишаються, що дозволяє продовжувати порівняльний аналіз результатів.

ДОДАТОК В

Текст програмних компонентів

Лістинг В.1 – Текст програмного компонента MainApp.java

```
package com.coursework.montecarlo.ui;
```

```
import com.coursework.montecarlo.algorithm.MonteCarloSimulation;
import com.coursework.montecarlo.algorithm.IMonteCarloAlgorithm;
import com.coursework.montecarlo.model.*;
import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.chart.LineChart;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.TextAlignment;
import javafx.stage.Stage;
import javafx.util.Duration;
```

```
public class MainApp extends Application {
    private static final int CANVAS_SIZE = 650;

    private final IMonteCarloAlgorithm simulation = new MonteCarloSimulation();
    private final ResultHistory history = new ResultHistory();
    private SimulationResult currentResult;
    private Timeline timeline;

    private Canvas canvas;
    private GraphicsContext gc;
    private TextField pointsField;
    private ComboBox<VisualizationMode> modeBox;
```

```

private Slider speedSlider;
private Label speedValueLabel;
private Button startButton;
private Button pauseButton;
private Button stepButton;
private Button resetButton;
private Button saveButton;
private Label piLabel;
private Label totalLabel;
private Label insideLabel;
private Label errorLabel;
private XYChart.Series<Number, Number> errorSeries;

```

```

@Override
public void start(Stage stage) {
    BorderPane root = new BorderPane();
    root.setPadding(new Insets(18));
    root.setStyle("-fx-background-color: #f7f8fb;");

    root.setLeft(createControlPanel());
    root.setCenter(createVisualizationArea());
    root.setRight(createResultsPanel());

    drawBaseArea();

    Scene scene = new Scene(root, 1600, 900);
    stage.setTitle("Метод Монте-Карло для оцінки значення числа  $\pi$ ");
    stage.setScene(scene);
    stage.setMaximized(true);
    stage.show();
}

```

```

private VBox createControlPanel() {
    Label title = createSectionTitle("ПАНЕЛЬ КЕРУВАННЯ");

    pointsField = new TextField("1000");
    pointsField.setPrefHeight(38);
    pointsField.setMaxWidth(Double.MAX_VALUE);
    pointsField.setStyle(inputStyle());

    modeBox = new ComboBox<>();
}

```

```

    modeBox.getItems().addAll(VisualizationMode.ANIMATED,
VisualizationMode.AUTOMATIC);
    modeBox.setValue(VisualizationMode.ANIMATED);
    modeBox.setMaxWidth(Double.MAX_VALUE);
    modeBox.setPrefHeight(38);
    modeBox.setStyle(inputStyle());

    speedSlider = new Slider(1, 100, 30);
    speedSlider.setShowTickMarks(false);
    speedSlider.setShowTickLabels(false);
    speedSlider.setMaxWidth(Double.MAX_VALUE);
    speedValueLabel = new Label("30");
    speedValueLabel.setStyle("-fx-font-size: 15px; -fx-text-fill: #1f2937; -fx-font-
weight: bold;");
    speedSlider.valueProperty().addListener((obs, oldValue, newValue) ->
        speedValueLabel.setText(String.valueOf(newValue.intValue()))
    );

    HBox speedTitle = new HBox(new Label("Швидкість анімації"), new Region(),
speedValueLabel);
    HBox.setHgrow(speedTitle.getChildren().get(1), Priority.ALWAYS);
    speedTitle.setAlignment(Pos.CENTER_LEFT);

    HBox speedText = new HBox();
    Label slowLabel = new Label("повільно");
    Label fastLabel = new Label("швидко");
    slowLabel.setStyle(smallTextStyle());
    fastLabel.setStyle(smallTextStyle());
    Region speedSpacer = new Region();
    HBox.setHgrow(speedSpacer, Priority.ALWAYS);
    speedText.getChildren().addAll(slowLabel, speedSpacer, fastLabel);

    startButton = createPrimaryButton("Запуск");
    pauseButton = createSecondaryButton("Пауза");
    stepButton = createSecondaryButton("Крок");
    resetButton = createSecondaryButton("Скидання");
    saveButton = createSecondaryButton("Збереження результатів");

    startButton.setOnAction(e -> startSimulation());
    pauseButton.setOnAction(e -> pauseSimulation());
    stepButton.setOnAction(e -> executeManualStep());

```



```

resetButton.setOnAction(e -> resetSimulation());
saveButton.setOnAction(e -> saveResult());
setInitialButtonState();

VBox box = new VBox(18,
    title,
    createFieldBlock("Введення кількості точок \n(Від 100 до 100000)",
pointsField),
    createFieldBlock("Вибір режиму візуалізації", modeBox),
    new VBox(8, speedTitle, speedSlider, speedText),
    createSpacer(22),
    startButton, pauseButton, stepButton, resetButton, saveButton
);
box.setPadding(new Insets(28, 28, 28, 28));
box.setPrefWidth(360);
box.setMinWidth(340);
box.setMaxWidth(380);
box.setAlignment(Pos.TOP_CENTER);
box.setStyle(panelStyle());
return box;
}

private VBox createFieldBlock(String labelText, Control control) {
    Label label = new Label(labelText);
    label.setStyle("-fx-font-size: 15px; -fx-text-fill: #1f2937;");
    VBox box = new VBox(8, label, control);
    box.setAlignment(Pos.CENTER_LEFT);
    box.setMaxWidth(Double.MAX_VALUE);
    return box;
}

private Button createPrimaryButton(String text) {
    Button button = new Button(text);
    button.setMaxWidth(Double.MAX_VALUE);
    button.setPrefHeight(48);
    button.setStyle("-fx-background-color: #0d6efd; -fx-text-fill: white; -fx-font-size:
16px; " +
        "-fx-font-weight: bold; -fx-background-radius: 7; -fx-cursor: hand;");
    return button;
}

```

```

private Button createSecondaryButton(String text) {
    Button button = new Button(text);
    button.setMaxWidth(Double.MAX_VALUE);
    button.setPrefHeight(46);
    button.setStyle("-fx-background-color: white; -fx-text-fill: #1f2937; -fx-font-size:
15px; " +
        "-fx-border-color: #d1d5db; -fx-border-radius: 7; -fx-background-radius: 7; -
fx-cursor: hand;");
    return button;
}

```

```

private Region createSpacer(double height) {
    Region spacer = new Region();
    spacer.setPrefHeight(height);
    return spacer;
}

```

```

private void setInitialButtonState() {
    startButton.setDisable(false);
    pauseButton.setDisable(true);
    stepButton.setDisable(true);
    resetButton.setDisable(true);
    saveButton.setDisable(true);
}

```

```

private void setAnimatedRunningButtonState() {
    startButton.setDisable(true);
    pauseButton.setDisable(false);
    stepButton.setDisable(true);
    resetButton.setDisable(true);
    saveButton.setDisable(true);
}

```

```

private void setPausedButtonState() {
    startButton.setDisable(true);
    pauseButton.setDisable(false);
    stepButton.setDisable(false);
    resetButton.setDisable(false);
    saveButton.setDisable(true);
}

```

```

private void setFinishedButtonState() {
    startButton.setDisable(true);
    pauseButton.setDisable(true);
    stepButton.setDisable(true);
    resetButton.setDisable(false);
    saveButton.setDisable(false);
}

private VBox createVisualizationArea() {
    Label title = createSectionTitle("ВІЗУАЛІЗАЦІЯ");
    canvas = new Canvas(CANVAS_SIZE, CANVAS_SIZE);
    gc = canvas.getGraphicsContext2D();

    Label caption = new Label("Графічна область моделювання");
    caption.setStyle("-fx-font-size: 16px; -fx-text-fill: #6b7280;");

    VBox pane = new VBox(18, title, canvas, caption);
    pane.setPadding(new Insets(28, 28, 28, 28));
    pane.setAlignment(Pos.TOP_CENTER);
    pane.setStyle(panelStyle());
    BorderPane.setMargin(pane, new Insets(0, 12, 0, 12));
    return pane;
}

private VBox createResultsPanel() {
    Label title = createSectionTitle("РЕЗУЛЬТАТИ ОБЧИСЛЕННЯ");
    Label resultsTitle = new Label("Результати обчислення");
    resultsTitle.setStyle("-fx-font-size: 15px; -fx-text-fill: #1f2937;");

    piLabel = createResultLabel(" $\pi \approx 0.000000$ ", true);
    totalLabel = createResultLabel("Кількість точок: 0", false);
    insideLabel = createResultLabel("У колі: 0", false);
    errorLabel = createResultLabel("Похибка: 0.000000", false);

    VBox resultsBox = new VBox(12, resultsTitle, piLabel, totalLabel, insideLabel,
errorLabel);
    resultsBox.setAlignment(Pos.CENTER);
    resultsBox.setMaxWidth(Double.MAX_VALUE);

    NumberAxis xAxis = new NumberAxis(100, 100000, 10000);
    NumberAxis yAxis = new NumberAxis();
    xAxis.setLabel("Кількість точок N");

```

```

yAxis.setLabel("Похибка");
xAxis.setForceZeroInRange(false);
yAxis.setForceZeroInRange(true);

```

```

LineChart<Number, Number> chart = new LineChart<>(xAxis, yAxis);
chart.setCreateSymbols(true);
chart.setAnimated(false);
chart.setLegendVisible(false);
chart.setPrefSize(620, 440);
chart.setMinSize(600, 420);
chart.setMaxSize(660, 470);

```

```

errorSeries = new XYChart.Series<>();
chart.getData().add(errorSeries);

```

```

Label chartCaption = new Label("Залежність похибки від кількості точок\n(за
збереженими результатами)");
chartCaption.setAlignment(Pos.CENTER);
chartCaption.setTextAlignment(TextAlignment.CENTER);
chartCaption.setStyle("-fx-font-size: 15px; -fx-text-fill: #1f2937;");

```

```

VBox box = new VBox(24, title, resultsBox, createSpacer(10), chart,
chartCaption);
box.setPadding(new Insets(28, 28, 28, 28));
box.setPrefWidth(650);
box.setMinWidth(620);
box.setAlignment(Pos.TOP_CENTER);
box.setStyle(panelStyle());
return box;
}

```

```

private Label createSectionTitle(String text) {
    Label label = new Label(text);
    label.setFont(Font.font("Arial", FontWeight.BOLD, 22));
    label.setStyle("-fx-text-fill: #111827;");
    label.setAlignment(Pos.CENTER);
    label.setMaxWidth(Double.MAX_VALUE);
    return label;
}

```

```

private Label createResultLabel(String text, boolean main) {

```

```

    Label label = new Label(text);
    label.setAlignment(Pos.CENTER);
    label.setMaxWidth(Double.MAX_VALUE);
    label.setTextAlignment(TextAlignment.CENTER);
    if (main) {
        label.setStyle("-fx-font-size: 23px; -fx-text-fill: #174ea6; -fx-font-weight:
bold;");
    } else {
        label.setStyle("-fx-font-size: 17px; -fx-text-fill: #1f2937;");
    }
    return label;
}

private String panelStyle() {
    return "-fx-background-color: white; -fx-background-radius: 10; -fx-border-radius:
10; " +
        "-fx-border-color: #e5e7eb; -fx-effect: dropshadow(gaussian,
rgba(0,0,0,0.08), 10, 0, 0, 2);";
}

private String inputStyle() {
    return "-fx-font-size: 15px; -fx-background-color: white; -fx-border-color:
#d1d5db; " +
        "-fx-border-radius: 6; -fx-background-radius: 6;";
}

private String smallTextStyle() {
    return "-fx-font-size: 13px; -fx-text-fill: #374151;";
}

private void startSimulation() {
    try {
        int totalPoints = Integer.parseInt(pointsField.getText().trim());
        SimulationParameters params = new SimulationParameters(totalPoints, (int)
speedSlider.getValue(), modeBox.getValue());

        simulation.initialize(params);
        currentResult = null;
        pauseButton.setText("Пайза");
        drawBaseArea();
        updateLabels(0, 0, 0, 0);
    }
}

```

```

        if (params.getMode() == VisualizationMode.AUTOMATIC) {
            runAutomaticMode();
            setFinishedButtonState();
        } else {
            startAnimatedMode(params);
            setAnimatedRunningButtonState();
        }
    } catch (NumberFormatException ex) {
        showError("Кількість точок має бути цілим числом.");
    } catch (IllegalArgumentException ex) {
        showError(ex.getMessage());
    }
}

private void runAutomaticMode() {
    while (!simulation.isFinished()) {
        IterationState state = simulation.nextStep();
        if (state.getPoint() != null) {
            drawPoint(state.getPoint());
        }
    }

    currentResult = simulation.createResult();
    updateLabels(currentResult.getTotalPoints(),
currentResult.getInsideCirclePoints(), currentResult.getPiEstimate(),
currentResult.getError());
}

private void startAnimatedMode(SimulationParameters params) {
    if (timeline != null) {
        timeline.stop();
    }
    double delay = Math.max(3, 900.0 / params.getAnimationSpeed());
    timeline = new Timeline(new KeyFrame(Duration.millis(delay), e ->
executeOneStep()));
    timeline.setCycleCount(Timeline.INDEFINITE);
    timeline.play();
}

private void executeOneStep() {

```

```

try {
    IterationState state = simulation.nextStep();
    if (state.getPoint() != null) {
        drawPoint(state.getPoint());
    }
    updateLabels(state.getGeneratedPoints(), state.getInsideCirclePoints(),
state.getPiEstimate(), state.getError());
    if (state.isFinished()) {
        if (timeline != null) {
            timeline.stop();
        }
        currentResult = simulation.createResult();
        updateLabels(currentResult.getTotalPoints(),
currentResult.getInsideCirclePoints(), currentResult.getPiEstimate(),
currentResult.getError());
        setFinishedButtonState();
    }
} catch (IllegalStateException ex) {
    showError("Спочатку натисніть кнопку 'Запуск'.");
}
}

private void executeManualStep() {
    if (timeline == null || timeline.getStatus() == Timeline.Status.RUNNING) {
        return;
    }

    executeOneStep();

    if (simulation.isFinished()) {
        currentResult = simulation.createResult();
        setFinishedButtonState();
    }
}

private void pauseSimulation() {
    if (timeline == null) {
        return;
    }
    if (timeline.getStatus() == Timeline.Status.RUNNING) {
        timeline.pause();
    }
}

```

```

        pauseButton.setText("Продовжити");
        setPausedButtonState();
    } else {
        timeline.play();
        pauseButton.setText("Пауза");
        setAnimatedRunningButtonState();
    }
}

private void resetSimulation() {
    if (timeline != null) {
        timeline.stop();
    }
    pauseButton.setText("Пауза");
    currentResult = null;
    drawBaseArea();
    updateLabels(0, 0, 0, 0);
    setInitialButtonState();
}

private void saveResult() {
    if (currentResult == null) {
        showError("Спочатку виконайте моделювання до кінця.");
        return;
    }
    history.addResult(currentResult);
    errorSeries.getData().add(new XYChart.Data<>(currentResult.getTotalPoints(),
currentResult.getError()));
}

private void drawBaseArea() {
    gc.setFill(Color.WHITE);
    gc.fillRect(0, 0, CANVAS_SIZE, CANVAS_SIZE);

    gc.setStroke(Color.web("#1f2937"));
    gc.setLineWidth(1.3);
    gc.strokeRect(0, 0, CANVAS_SIZE, CANVAS_SIZE);

    gc.setStroke(Color.web("#6b7280"));
    gc.setLineWidth(1.5);
    gc.strokeOval(0, 0, CANVAS_SIZE, CANVAS_SIZE);
}

```



```

    }

    private void drawPoint(Point point) {
        double px = point.getX() * CANVAS_SIZE;
        double py = CANVAS_SIZE - point.getY() * CANVAS_SIZE;
        gc.setFill(point.isInsideCircle() ? Color.web("#0d6efd") : Color.web("#ff3b30"));
        gc.fillOval(px - 1.35, py - 1.35, 2.7, 2.7);
    }

    private void updateLabels(int total, int inside, double pi, double error) {
        piLabel.setText(String.format("π ≈ %.9f", pi));
        totalLabel.setText("Кількість точок: " + total);
        insideLabel.setText("У колі: " + inside);
        errorLabel.setText(String.format("Похибка: %.10f", error));
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Помилка");
        alert.setHeaderText("Некоректні дані");
        alert.setContentText(message);
        alert.showAndWait();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Лістинг В.2 – Текст програмного компонента IMonteCarloAlgorithm.java

```

package com.coursework.montecarlo.algorithm;

import com.coursework.montecarlo.model.IterationState;
import com.coursework.montecarlo.model.SimulationParameters;
import com.coursework.montecarlo.model.SimulationResult;

public interface IMonteCarloAlgorithm {
    void initialize(SimulationParameters parameters);
    IterationState nextStep();
    boolean isFinished();
}

```

```
SimulationResult createResult();
}
```

Лістинг В.3 – Текст програмного компонента MonteCarloSimulation.java

```
package com.coursework.montecarlo.algorithm;

import com.coursework.montecarlo.model.*;

import java.util.Random;

public class MonteCarloSimulation implements IMonteCarloAlgorithm {
    private final Random random = new Random();
    private final Statistics statistics = new Statistics();
    private SimulationParameters parameters;
    private long startTime;

    @Override
    public void initialize(SimulationParameters parameters) {
        this.parameters = parameters;
        this.statistics.reset();
        this.startTime = System.currentTimeMillis();
    }

    @Override
    public IterationState nextStep() {
        ensureInitialized();

        if (isFinished()) {
            return createState(null, true);
        }

        Point point = generatePoint();
        statistics.update(point);
        return createState(point, isFinished());
    }

    @Override
    public boolean isFinished() {
        return parameters != null && statistics.getGeneratedPoints() >=
parameters.getTotalPoints();
    }
}
```

```

    }

    @Override
    public SimulationResult createResult() {
        return new SimulationResult(
            statistics.getGeneratedPoints(),
            statistics.getInsideCirclePoints(),
            statistics.calculatePi(),
            statistics.calculateError(),
            System.currentTimeMillis() - startTime
        );
    }

    private Point generatePoint() {
        return new Point(random.nextDouble(), random.nextDouble());
    }

    private IterationState createState(Point point, boolean finished) {
        return new IterationState(
            point,
            statistics.getGeneratedPoints(),
            statistics.getInsideCirclePoints(),
            statistics.calculatePi(),
            statistics.calculateError(),
            finished
        );
    }

    private void ensureInitialized() {
        if (parameters == null) {
            throw new IllegalStateException("Параметри моделювання не ініціалізовано.");
        }
    }
}

```

Лістинг В.4 – Текст програмного компонента Point.java

```

package com.coursework.montecarlo.model;

public class Point {

```

```

private final double x;
private final double y;

public Point(double x, double y) {
    this.x = x;
    this.y = y;
}

public double getX() {
    return x;
}

public double getY() {
    return y;
}

public boolean isInsideCircle() {
    double dx = x - 0.5;
    double dy = y - 0.5;
    return dx * dx + dy * dy <= 0.25;
}
}

```

Лістинг В.5 – Текст програмного компонента IterationState.java

```

package com.coursework.montecarlo.model;

public class IterationState {
    private final Point point;
    private final int generatedPoints;
    private final int insideCirclePoints;
    private final double piEstimate;
    private final double error;
    private final boolean finished;

    public IterationState(Point point, int generatedPoints, int insideCirclePoints,
        double piEstimate, double error, boolean finished) {
        this.point = point;
        this.generatedPoints = generatedPoints;
        this.insideCirclePoints = insideCirclePoints;
        this.piEstimate = piEstimate;
    }
}

```

```

        this.error = error;
        this.finished = finished;
    }

    public Point getPoint() { return point; }
    public int getGeneratedPoints() { return generatedPoints; }
    public int getInsideCirclePoints() { return insideCirclePoints; }
    public double getPiEstimate() { return piEstimate; }
    public double getError() { return error; }
    public boolean isFinished() { return finished; }
}

```

Лістинг В.6 – Текст програмного компонента ResultHistory.java

```

package com.coursework.montecarlo.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ResultHistory {
    private final List<SimulationResult> results = new ArrayList<>();

    public void addResult(SimulationResult result) {
        results.add(result);
    }

    public void clear() {
        results.clear();
    }

    public List<SimulationResult> getResults() {
        return Collections.unmodifiableList(results);
    }
}

```

Лістинг В.7 – Текст програмного компонента SimulationParameters.java

```

package com.coursework.montecarlo.model;

```

```

public class SimulationParameters {
    private final int totalPoints;
    private final int animationSpeed;
    private final VisualizationMode mode;

    public SimulationParameters(int totalPoints, int animationSpeed, VisualizationMode
mode) {
        if (totalPoints < 100 || totalPoints > 100000) {
            throw new IllegalArgumentException("Кількість точок має бути від 100 до
100000.");
        }
        this.totalPoints = totalPoints;
        this.animationSpeed = animationSpeed;
        this.mode = mode;
    }

    public int getTotalPoints() {
        return totalPoints;
    }

    public int getAnimationSpeed() {
        return animationSpeed;
    }

    public VisualizationMode getMode() {
        return mode;
    }
}

```

Лістинг В.8 – Текст програмного компонента SimulationResult.java

```

package com.coursework.montecarlo.model;

public class SimulationResult {
    private final int totalPoints;
    private final int insideCirclePoints;
    private final double piEstimate;
    private final double error;
    private final long runtimeMillis;

    public SimulationResult(int totalPoints, int insideCirclePoints, double piEstimate,

```

```

double error, long runtimeMillis) {
    this.totalPoints = totalPoints;
    this.insideCirclePoints = insideCirclePoints;
    this.piEstimate = piEstimate;
    this.error = error;
    this.runtimeMillis = runtimeMillis;
}

public int getTotalPoints() { return totalPoints; }
public int getInsideCirclePoints() { return insideCirclePoints; }
public double getPiEstimate() { return piEstimate; }
public double getError() { return error; }
public long getRuntimeMillis() { return runtimeMillis; }
}

```

ЛІСТИНГ В.9 – Текст програмного компонента Statistics.java

```

package com.coursework.montecarlo.model;

public class Statistics {
    public static final double TRUE_PI = 3.1415926535;

    private int generatedPoints;
    private int insideCirclePoints;

    public void reset() {
        generatedPoints = 0;
        insideCirclePoints = 0;
    }

    public void update(Point point) {
        generatedPoints++;
        if (point.isInsideCircle()) {
            insideCirclePoints++;
        }
    }

    public int getGeneratedPoints() {
        return generatedPoints;
    }
}

```

```

public int getInsideCirclePoints() {
    return insideCirclePoints;
}

public double calculatePi() {
    if (generatedPoints == 0) {
        return 0.0;
    }
    return 4.0 * insideCirclePoints / generatedPoints;
}

public double calculateError() {
    if (generatedPoints == 0) {
        return 0.0;
    }
    return Math.abs(calculatePi() - TRUE_PI);
}
}

```

Лістинг В.10 – Текст програмного компонента VisualizationMode.java

```

package com.coursework.montecarlo.model;

```

```

public enum VisualizationMode {
    AUTOMATIC("Автоматичний"),
    ANIMATED("Анімований");

    private final String title;

    VisualizationMode(String title) {
        this.title = title;
    }

    @Override
    public String toString() {
        return title;
    }
}

```